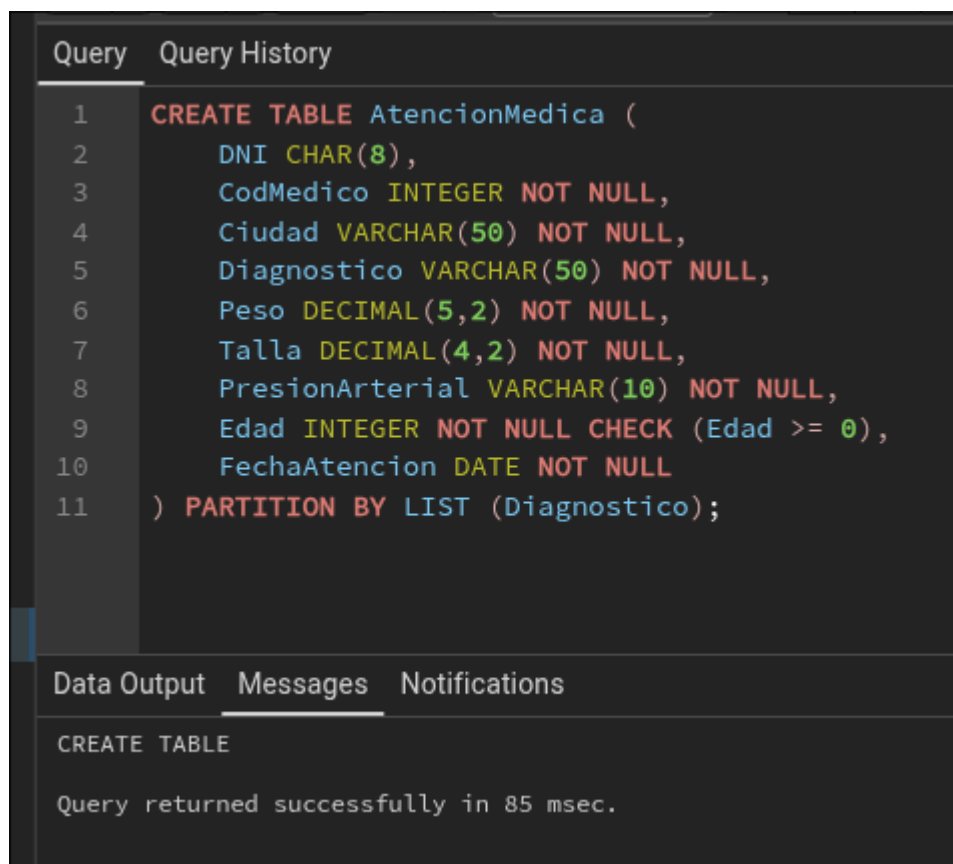


Laboratorio 13: Fragmentación Dinámica, Asignación y Consultas Distribuidas

P1. Creación de particiones

Creamos la tabla AtencionMedica con la partición por el atributo 'Diagnostico'

```
CREATE TABLE AtencionMedica (  
    DNI CHAR(8),  
    CodMedico INTEGER NOT NULL,  
    Ciudad VARCHAR(50) NOT NULL,  
    Diagnostico VARCHAR(50) NOT NULL,  
    Peso DECIMAL(5,2) NOT NULL,  
    Talla DECIMAL(4,2) NOT NULL,  
    PresionArterial VARCHAR(10) NOT NULL,  
    Edad INTEGER NOT NULL CHECK (Edad >= 0),  
    FechaAtencion DATE NOT NULL  
) PARTITION BY LIST (Diagnostico);
```



The screenshot shows a SQL IDE interface with a dark theme. The top panel has tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL script for creating a table named 'AtencionMedica' with various attributes and a partitioning clause. The script is as follows:

```
1 CREATE TABLE AtencionMedica (  
2     DNI CHAR(8),  
3     CodMedico INTEGER NOT NULL,  
4     Ciudad VARCHAR(50) NOT NULL,  
5     Diagnostico VARCHAR(50) NOT NULL,  
6     Peso DECIMAL(5,2) NOT NULL,  
7     Talla DECIMAL(4,2) NOT NULL,  
8     PresionArterial VARCHAR(10) NOT NULL,  
9     Edad INTEGER NOT NULL CHECK (Edad >= 0),  
10    FechaAtencion DATE NOT NULL  
11 ) PARTITION BY LIST (Diagnostico);
```

The bottom panel has tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the following output:

```
CREATE TABLE  
  
Query returned successfully in 85 msec.
```

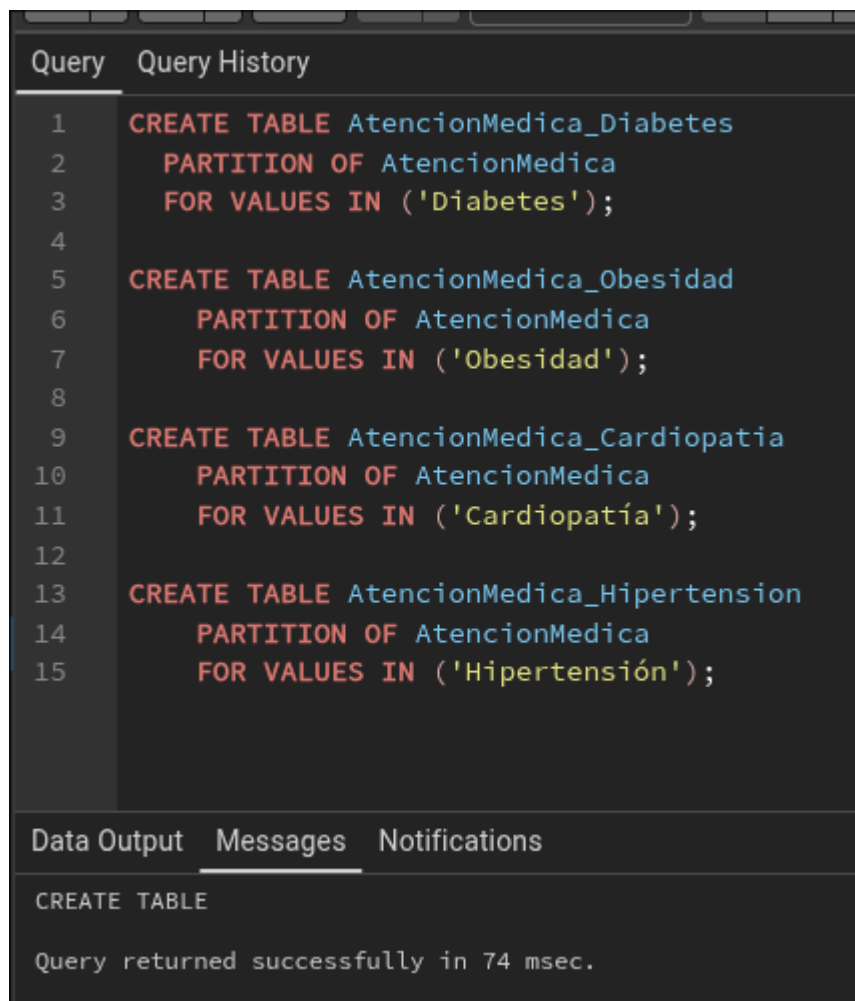
Creamos las particiones por los diferentes tipos de diagnósticos.

```
CREATE TABLE AtencionMedica_Diabetes  
PARTITION OF AtencionMedica  
FOR VALUES IN ('Diabetes');
```

```
CREATE TABLE AtencionMedica_Obesidad
PARTITION OF AtencionMedica
FOR VALUES IN ('Obesidad');

CREATE TABLE AtencionMedica_Cardiopatía
PARTITION OF AtencionMedica
FOR VALUES IN ('Cardiopatía');

CREATE TABLE AtencionMedica_Hipertension
PARTITION OF AtencionMedica
FOR VALUES IN ('Hipertensión');
```



```
Query  Query History
1  CREATE TABLE AtencionMedica_Diabetes
2  PARTITION OF AtencionMedica
3  FOR VALUES IN ('Diabetes');
4
5  CREATE TABLE AtencionMedica_Obesidad
6  PARTITION OF AtencionMedica
7  FOR VALUES IN ('Obesidad');
8
9  CREATE TABLE AtencionMedica_Cardiopatía
10 PARTITION OF AtencionMedica
11 FOR VALUES IN ('Cardiopatía');
12
13 CREATE TABLE AtencionMedica_Hipertension
14 PARTITION OF AtencionMedica
15 FOR VALUES IN ('Hipertensión');

Data Output  Messages  Notifications
CREATE TABLE
Query returned successfully in 74 msec.
```

Llenamos la tabla con 60k registros sinteticos:

```
INSERT INTO AtencionMedica
(DNI, CodMedico, Ciudad, Diagnostico, Peso, Talla, PresionArterial, Edad,
FechaAtencion)
SELECT
LPAD(((random() * 49999)::int + 1)::text, 8, '0') AS DNI,
(random() * 199 + 1)::int AS
CodMedico,
(ARRAY['Lima', 'Callao', 'Arequipa', 'Cusco', 'Trujillo', 'Piura',
'Chiclayo', 'Iquitos', 'Huancayo', 'Tacna', 'Pucallpa', 'Juliaca',
```

```
        'Ica','Ayacucho','Huaraz','Tumbes','Moquegua','Chimbote'])
    [ (random() * 17 + 1)::int ] AS Ciudad,
    (ARRAY['Diabetes','Obesidad','Cardiopatía','Hipertensión'])
    [ (random() * 3 + 1)::int ] AS
    Diagnostico,
    (random() * 60 + 40)::numeric(5,2) AS Peso,
    (random() * 0.60 + 1.40)::numeric(4,2) AS Talla,
    ((random() * 60 + 90)::int) || '/' || ((random() * 40 + 60)::int) AS
    PresionArterial,
    (random() * 95)::int AS Edad,
    DATE '2024-01-01' + ((random() * 730)::int) AS
    FechaAtencion
FROM generate_series(1, 60000);
```

QueryQuery History

1

INSERT INTO AtencionMedica

2

(DNI, CodMedico, Ciudad, Diagnostico, Peso, Talla, PresionArterial, Edad, FechaAtencion)

3

SELECT

4

LPAD(((random() * 49999)::int + 1)::text, 8, '0')

AS DNI,

5

(random() * 199 + 1)::int

AS CodMedico,

6

(ARRAY['Lima','Callao','Arequipa','Cusco','Trujillo','Piura',

7

'Chiclayo','Iquitos','Huancayo','Tacna','Pucallpa','Juliaca',

8

'Ica','Ayacucho','Huaraz','Tumbes','Moquegua','Chimbote'])

AS Ciudad,

9

[(random() * 17 + 1)::int]

AS Diagnostico,

10

(ARRAY['Diabetes','Obesidad','Cardiopatía','Hipertensión'])

AS Peso,

11

[(random() * 3 + 1)::int]

AS Talla,

12

(random() * 60 + 40)::numeric(5,2)

AS PresionArterial,

13

(random() * 0.60 + 1.40)::numeric(4,2)

AS Edad,

14

((random() * 60 + 90)::int) || '/' || ((random() * 40 + 60)::int)

AS FechaAtencion

15

(random() * 95)::int

AS

16

DATE '2024-01-01' + ((random() * 730)::int)

AS

17

FROM generate_series(1, 60000);

Data OutputMessagesNotifications

INSERT 0 60000

Query returned successfully in 210 msec.

Verificamos:

QueryQuery History

1

SELECT COUNT(*) FROM AtencionMedica;

Data OutputMessagesNotifications

≡+

▼

▼

SQL

count bigint

160000

3 / 46

P2

En esta sección se implementó un mecanismo para que la tabla `AtencionMedica` pueda crear nuevas particiones automáticamente cuando se registre una atención con un diagnóstico que todavía no cuenta con un fragmento propio.

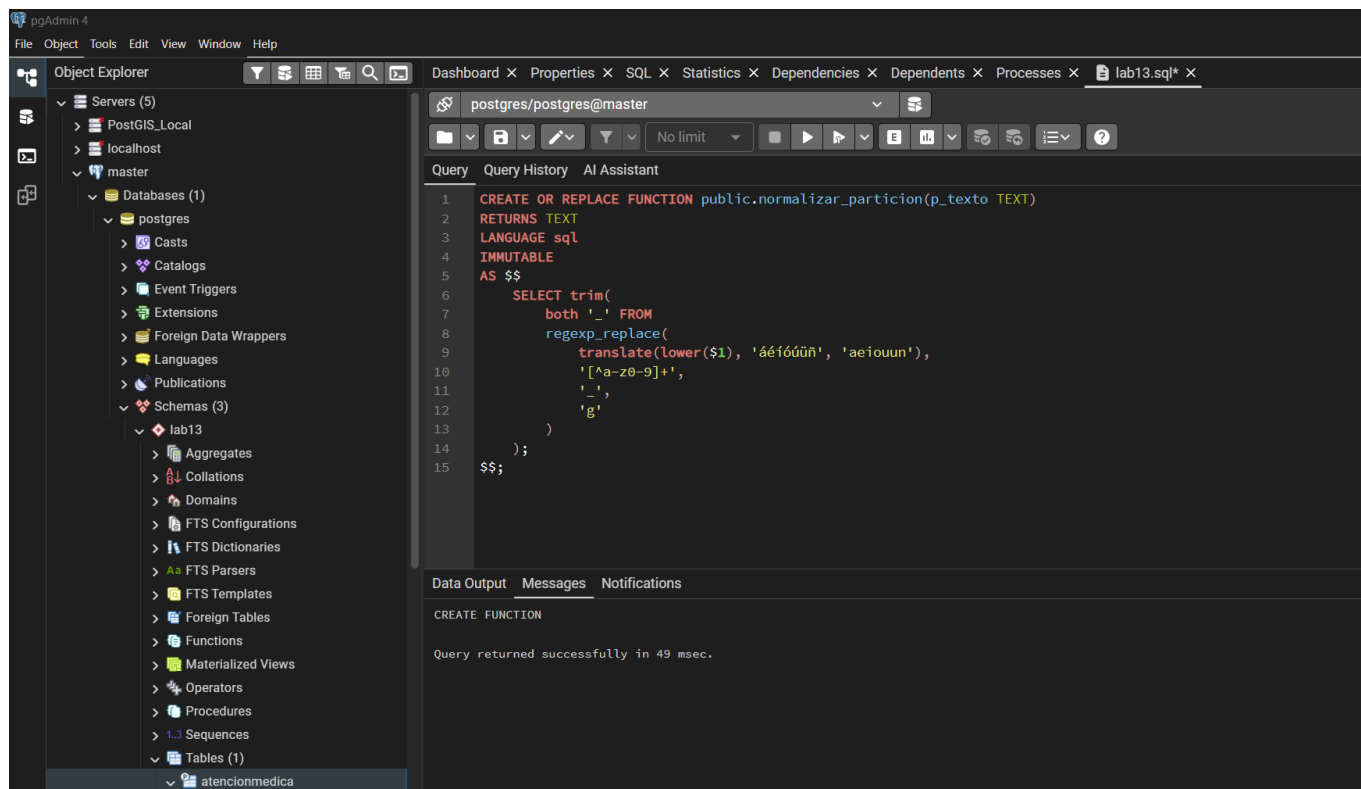
Inicialmente, la tabla solo posee particiones para Diabetes, Obesidad, Cardiopatía e Hipertensión. Sin embargo, pueden aparecer nuevos diagnósticos durante el registro de atenciones. Por ello, se desarrolló una función auxiliar y un procedimiento almacenado que permiten identificar, crear y reutilizar particiones de forma dinámica.

Función para normalizar el nombre de las particiones

Primero, se creó la función `normalizar_particion`. Su objetivo es transformar el nombre de un diagnóstico en un nombre válido para una tabla de PostgreSQL.

La función convierte el texto a minúsculas, elimina tildes y reemplaza espacios o caracteres especiales por guiones bajos. Por ejemplo, un diagnóstico como `Insuficiencia Renal` se transformaría en `insuficiencia_renal`. Esto permite generar nombres consistentes para las particiones dinámicas.

```
CREATE OR REPLACE FUNCTION public.normalizar_particion(p_texto TEXT)
RETURNS TEXT
LANGUAGE sql
IMMUTABLE
AS $$
    SELECT trim(
        both '_' FROM
        regexp_replace(
            translate(lower($1), 'áéíóúüñ', 'aeiouun'),
            '^[^a-z0-9]+',
            '-',
            'g'
        )
    );
$$;
```



Procedimiento para insertar atenciones y crear fragmentos

Luego, se implementó el procedimiento almacenado `insertar_atencion_dinamica`. Este procedimiento recibe todos los datos de una nueva atención médica, incluido el diagnóstico.

Primero, construye el nombre que tendrá la posible partición usando el prefijo `atencionmedica_` y la función de normalización. Por ejemplo, para el diagnóstico `Asma`, el procedimiento genera el nombre `atencionmedica_asma`.

Después, mediante `to_regclass`, verifica si ya existe una tabla con dicho nombre. Si no existe, se utiliza SQL dinámico con `EXECUTE format(...)` para crear una nueva partición de la tabla `atencionmedica`, asociada exclusivamente a ese diagnóstico.

Finalmente, el procedimiento inserta la atención en la tabla padre. PostgreSQL identifica automáticamente la partición correspondiente y almacena el registro en ella.

```

CREATE OR REPLACE PROCEDURE public.insertar_atencion_dinamica(
    p_dni CHAR(8),
    p_codmedico INTEGER,
    p_ciudad VARCHAR(50),
    p_diagnostico VARCHAR(50),
    p_peso DECIMAL(5,2),
    p_talla DECIMAL(4,2),
    p_presionarterial VARCHAR(10),
    p_edad INTEGER,
    p_fechaatencion DATE
)
LANGUAGE plpgsql
AS $$
DECLARE

```

```

v_particion TEXT;
BEGIN
v_particion := 'atencionmedica_' ||
               public.normalizar_particion(p_diagnostico);

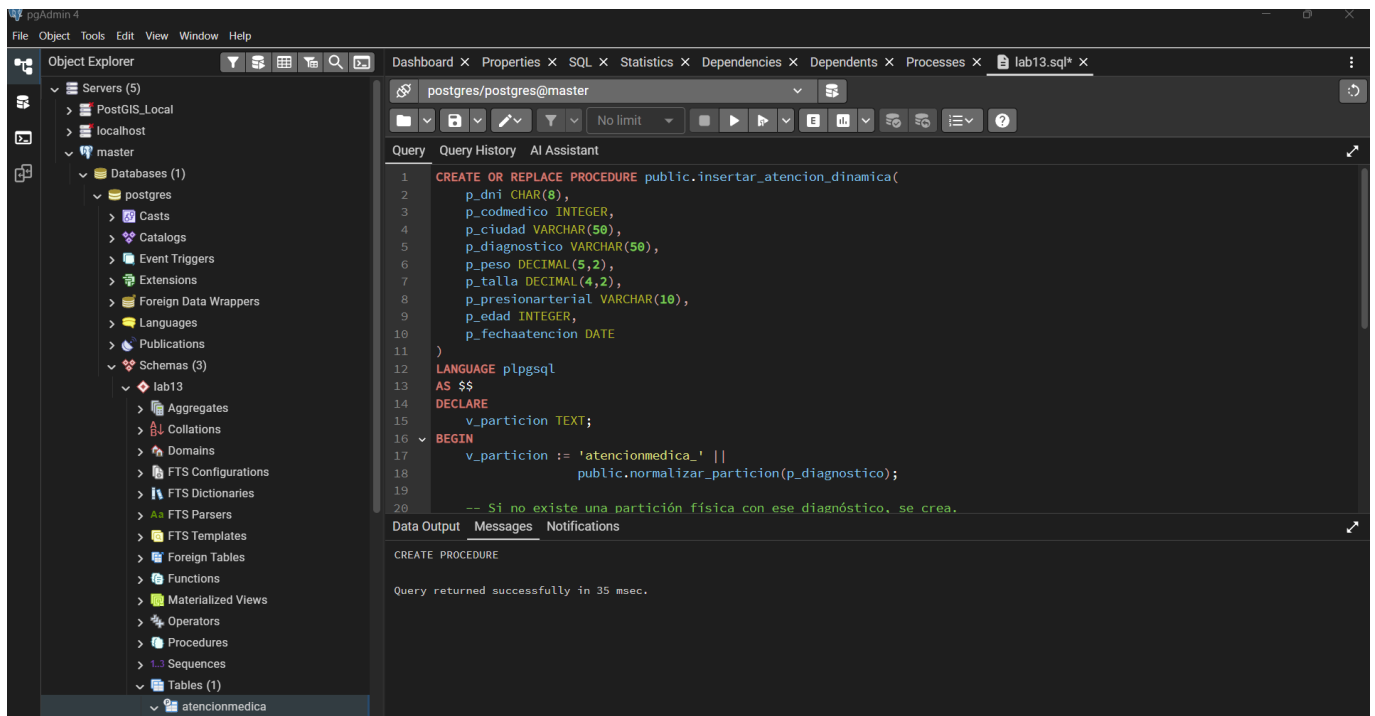
-- Si no existe una partición física con ese diagnóstico, se crea.
IF to_regclass(format('public.%I', v_particion)) IS NULL THEN

    EXECUTE format(
        'CREATE TABLE public.%I
        PARTITION OF public.atencionmedica
        FOR VALUES IN (%L)',
        v_particion,
        p_diagnostico
    );

    RAISE NOTICE 'Partición creada dinámicamente: %', v_particion;
END IF;

INSERT INTO public.atencionmedica
(dni, codmedico, ciudad, diagnostico, peso, talla,
presionararterial, edad, fechaatencion)
VALUES
(p_dni, p_codmedico, p_ciudad, p_diagnostico, p_peso, p_talla,
p_presionararterial, p_edad, p_fechaatencion);
END;
$$;

```



Inserción de registros con nuevos diagnósticos

Para probar el funcionamiento del procedimiento, se insertaron registros con diagnósticos que no estaban contemplados inicialmente: Asma, Anemia, Gastritis y Dengue.

Al ejecutar los primeros cuatro llamados, el procedimiento detecta que no existen particiones para dichos diagnósticos y crea automáticamente las tablas correspondientes. Posteriormente, se inserta un segundo registro con diagnóstico Asma para comprobar que el procedimiento reutiliza la partición ya creada, sin generar una tabla duplicada.

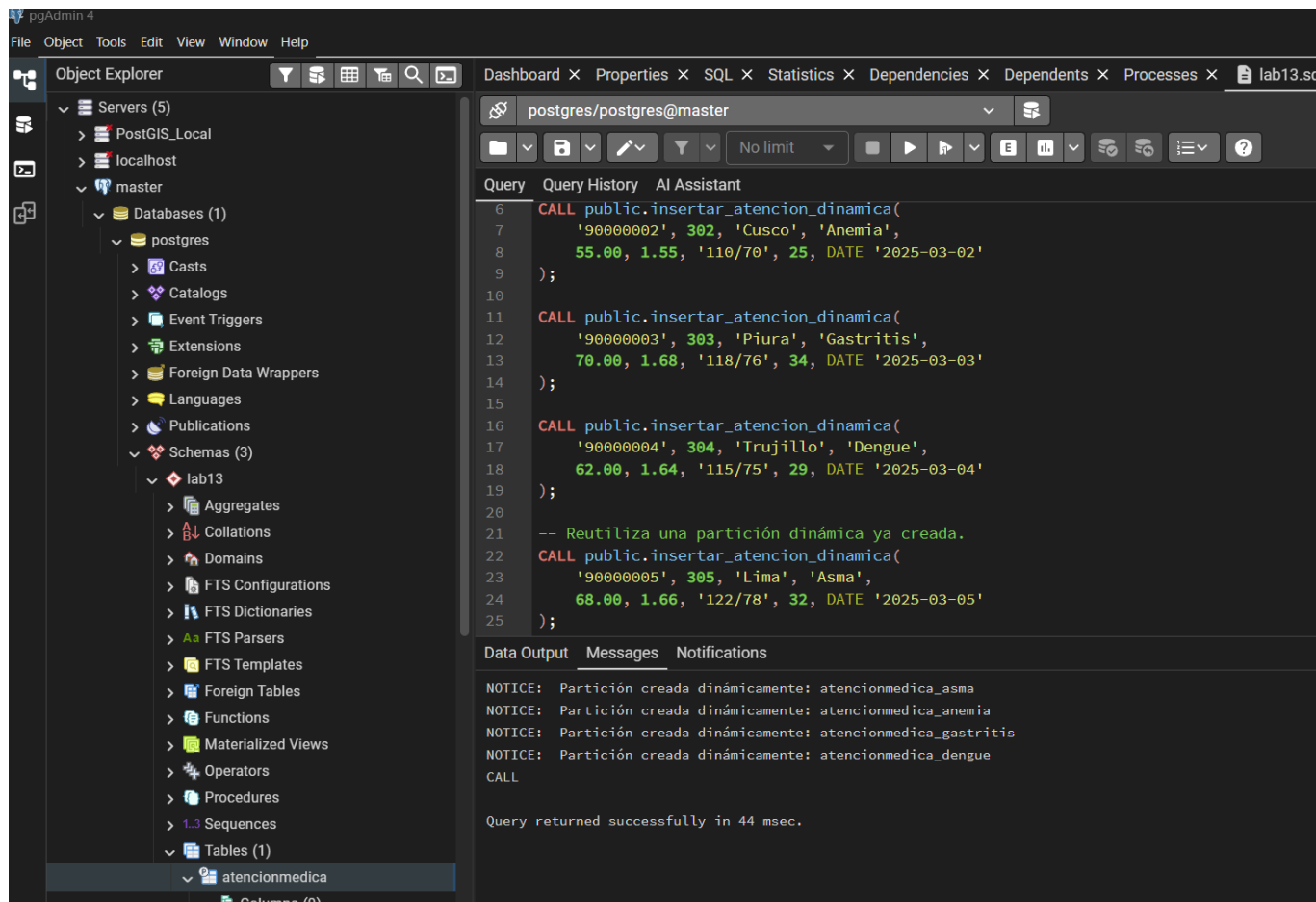
```
CALL public.insertar_atencion_dinamica(
  '90000001', 301, 'Arequipa', 'Asma',
  64.00, 1.60, '120/80', 30, DATE '2025-03-01'
);

CALL public.insertar_atencion_dinamica(
  '90000002', 302, 'Cusco', 'Anemia',
  55.00, 1.55, '110/70', 25, DATE '2025-03-02'
);

CALL public.insertar_atencion_dinamica(
  '90000003', 303, 'Piura', 'Gastritis',
  70.00, 1.68, '118/76', 34, DATE '2025-03-03'
);

CALL public.insertar_atencion_dinamica(
  '90000004', 304, 'Trujillo', 'Dengue',
  62.00, 1.64, '115/75', 29, DATE '2025-03-04'
);

-- Reutiliza una partición dinámica ya creada.
CALL public.insertar_atencion_dinamica(
  '90000005', 305, 'Lima', 'Asma',
  68.00, 1.66, '122/78', 32, DATE '2025-03-05'
);
```



Verificación de las particiones creadas

Finalmente, se utilizó la columna del sistema tableoid para identificar la tabla física en la que fue almacenado cada registro. La conversión tableoid::regclass permite mostrar directamente el nombre de la partición.

```
SELECT
    tableoid::regclass AS partition,
    diagnostico,
    COUNT(*) AS registros
FROM public.atencionmedica
WHERE dni IN ('90000001', '90000002', '90000003', '90000004', '90000005')
GROUP BY tableoid::regclass, diagnostico
ORDER BY partition;
```

El resultado evidencia que se crearon las particiones atencionmedica_asma, atencionmedica_anemia, atencionmedica_gastritis y atencionmedica_dengue. Asimismo, la partición atencionmedica_asma contiene dos registros, lo cual confirma que, al insertar nuevamente una atención con un diagnóstico existente, el procedimiento reutiliza la partición creada previamente.

De esta manera, se logra una fragmentación horizontal dinámica, ya que el sistema puede incorporar nuevos diagnósticos sin requerir la creación manual de una partición por parte del administrador.

partition regclass	diagnostico character varying (50)	registros bigint
atencionmedica_asma	Asma	2
atencionmedica_anemia	Anemia	1
atencionmedica_gastri...	Gastritis	1
atencionmedica_dengue	Dengue	1

Investigación: uso de triggers en tablas particionadas

PostgreSQL permite definir triggers sobre tablas particionadas. Cuando se crea un trigger por fila en la tabla padre, PostgreSQL genera triggers equivalentes en las particiones existentes y también en las particiones que se creen posteriormente.

Sin embargo, para este problema no se empleó un trigger como mecanismo principal de creación dinámica de particiones, porque al insertar una fila en una tabla particionada PostgreSQL debe determinar primero la partición destino según el valor de la clave de partición. Si no existe una partición compatible con dicho valor, la inserción produce un error.

Los triggers podrían utilizarse para tareas complementarias, como auditoría, validación de valores o registro de eventos. Sin embargo, en este laboratorio se utilizó un procedimiento almacenado porque permite verificar explícitamente la existencia de la partición antes de ejecutar el **INSERT**.

El procedimiento **insertar_atencion_dinamica** construye el nombre de la partición, verifica su existencia mediante **to_regclass**, la crea dinámicamente con **CREATE TABLE ... PARTITION OF** cuando es necesario y finalmente inserta el registro en la tabla padre. De esta manera, al momento de la inserción PostgreSQL ya dispone de una partición válida para enrutar la atención médica.

Prueba de comportamiento de un trigger en una tabla particionada

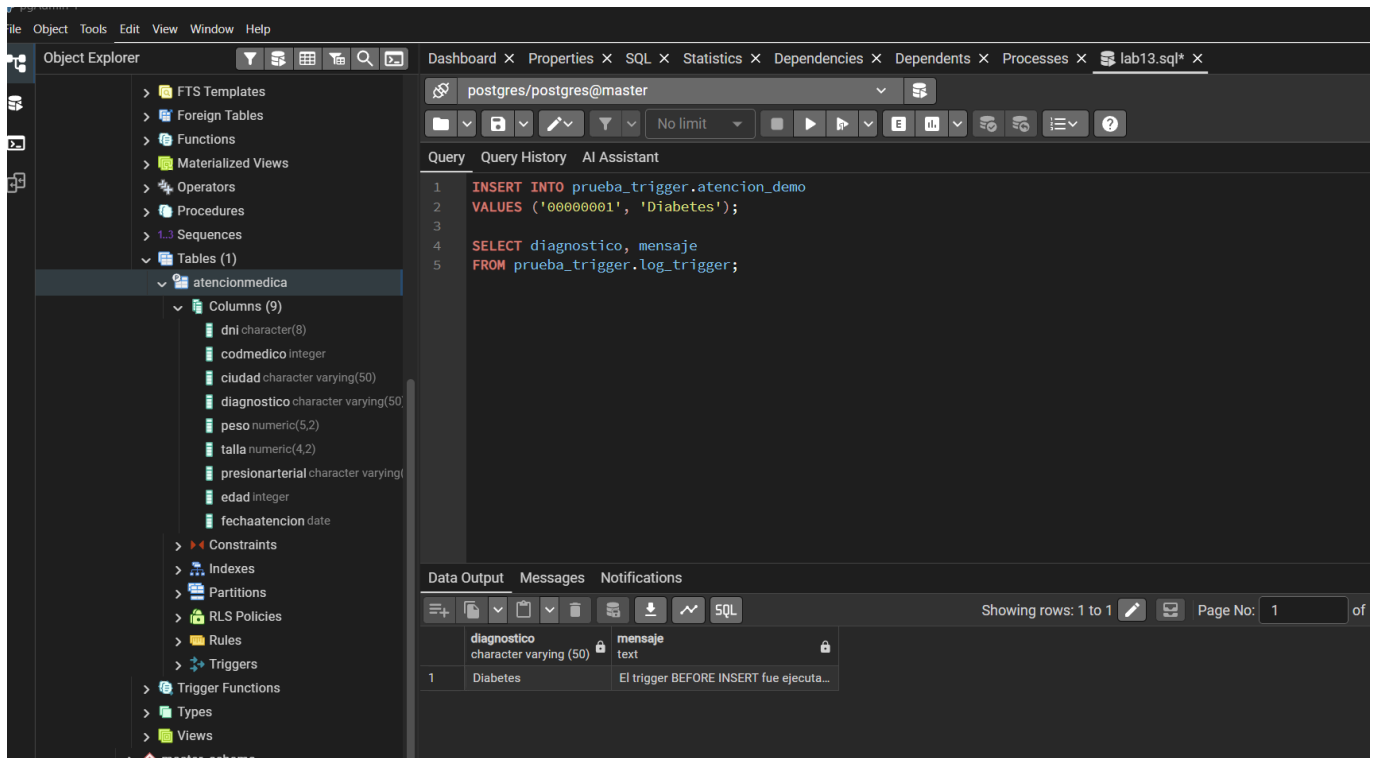
Para evidenciar este comportamiento, se creó un esquema de prueba independiente, una tabla particionada con una única partición para el diagnóstico **Diabetes**, una tabla de auditoría y un trigger **BEFORE INSERT**.

```
CREATE SCHEMA IF NOT EXISTS prueba_trigger;
```

```
CREATE TABLE prueba_trigger.log_trigger (  
    id SERIAL PRIMARY KEY,  
    diagnostico VARCHAR(50),  
    mensaje TEXT,  
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE prueba_trigger.atencion_demo (  
    dni CHAR(8),  
    diagnostico VARCHAR(50) NOT NULL  
) PARTITION BY LIST (diagnostico);  
  
CREATE TABLE prueba_trigger.atencion_demo_diabetes  
PARTITION OF prueba_trigger.atencion_demo  
FOR VALUES IN ('Diabetes');  
  
CREATE OR REPLACE FUNCTION prueba_trigger.registrar_trigger()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO prueba_trigger.log_trigger (  
        diagnostico,  
        mensaje  
    )  
    VALUES (  
        NEW.diagnostico,  
        'El trigger BEFORE INSERT fue ejecutado'  
    );  
  
    RETURN NEW;  
END;  
$$;  
  
CREATE TRIGGER trg_before_insert_demo  
BEFORE INSERT ON prueba_trigger.atencion_demo  
FOR EACH ROW  
EXECUTE FUNCTION prueba_trigger.registrar_trigger();
```

Primero, se insertó un registro con diagnóstico **Diabetes**, para el cual sí existe una partición compatible. Posteriormente, se consultó la tabla de auditoría.

```
INSERT INTO prueba_trigger.atencion_demo  
VALUES ('00000001', 'Diabetes');  
  
SELECT *  
FROM prueba_trigger.log_trigger;
```

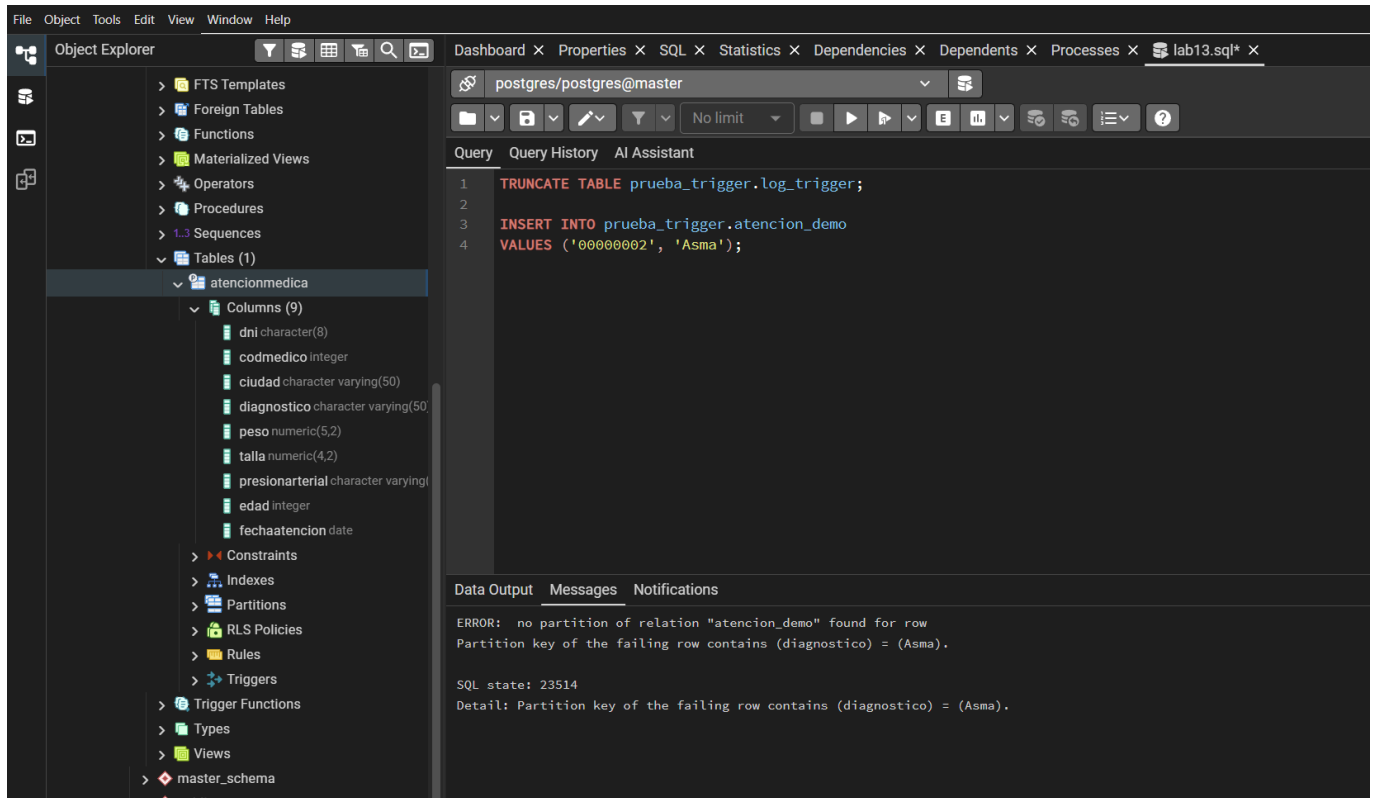


El resultado muestra que el trigger fue ejecutado correctamente, ya que se registró una fila de auditoría para el diagnóstico **Diabetes**. Esto ocurre porque PostgreSQL encontró la partición **atencion_demo_diabetes** y pudo enrutar el registro antes de realizar la inserción.

Luego, se vació la tabla de auditoría y se intentó insertar un diagnóstico que no posee una partición asociada.

```
TRUNCATE TABLE prueba_trigger.log_trigger;

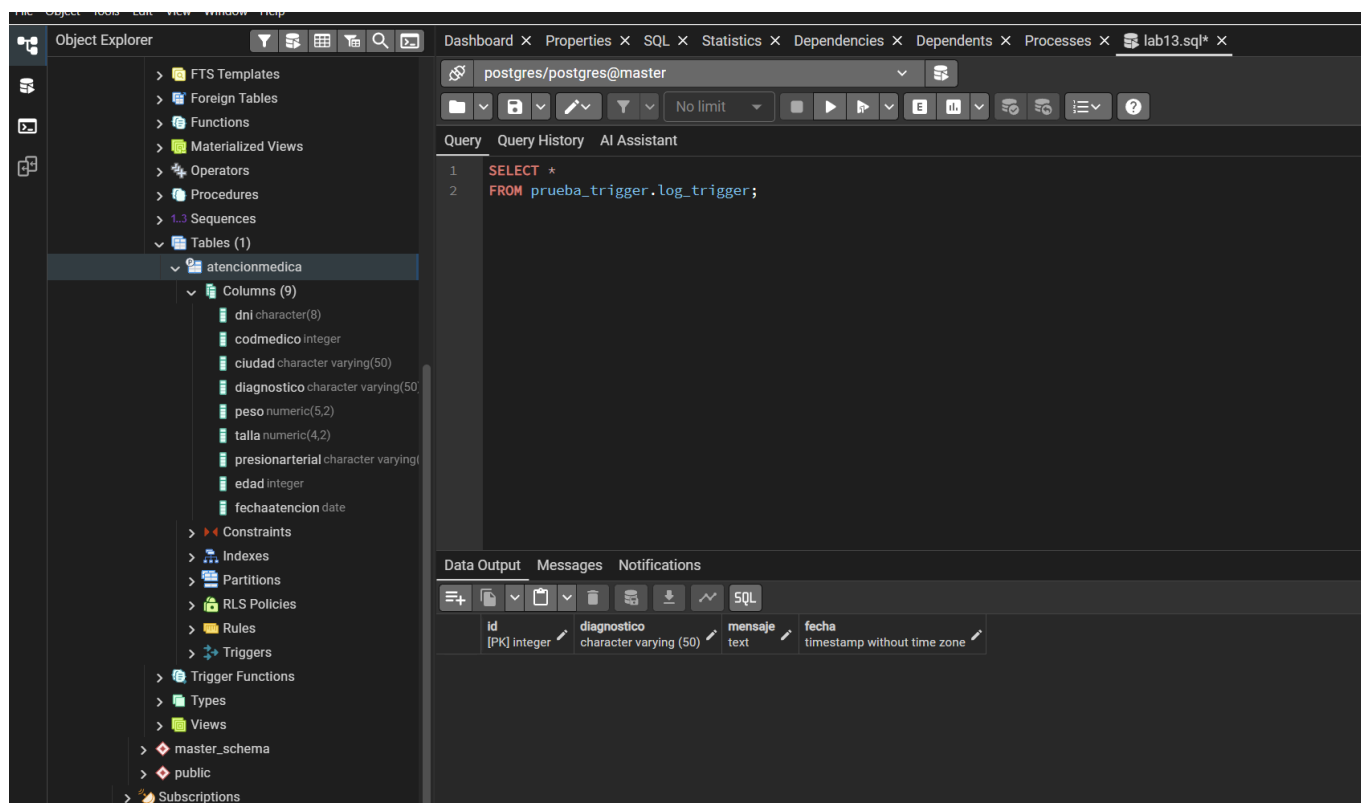
INSERT INTO prueba_trigger.atencion_demo
VALUES ('00000002', 'Asma');
```



La inserción produjo el error `no partition of relation "atencion_demo" found for row`. El mensaje indica que el valor `Asma` no cuenta con una partición compatible, por lo que PostgreSQL no puede enrutar el registro hacia una tabla física.

Finalmente, se verificó el contenido de la tabla de auditoría después del intento fallido.

```
SELECT *  
FROM prueba_trigger.log_trigger;
```



La consulta no devuelve filas, lo cual evidencia que el intento de inserción con **Asma** no llegó a registrar la ejecución del trigger. Por tanto, aunque los triggers pueden utilizarse en tablas particionadas, no constituyen una solución directa para crear una nueva partición cuando todavía no existe un destino válido para la fila.

Por esta razón, el procedimiento **insertar_atencion_dinamica** resulta más adecuado para el laboratorio: primero verifica y crea la partición necesaria, y luego inserta el registro en la tabla padre.

P3. Asignación distribuida

Para implementar la distribución de fragmentos se creó una red Docker exclusiva denominada **lab12-net**. En esta red se desplegaron tres instancias PostgreSQL: un servidor coordinador denominado **lab12-master** y dos servidores remotos, **lab12-worker1** y **lab12-worker2**.

El servidor master gestiona la tabla particionada y coordina las consultas. Los workers almacenan físicamente fragmentos de la tabla **AtencionMedica**. La comunicación se implementa con **postgres_fdw**, mientras que **dblink** permite crear dinámicamente nuevas tablas en los servidores remotos.

Creación del entorno Docker

```
docker network create lab12-net
```

```
docker run -d --name lab12-master --network lab12-net -e POSTGRES_PASSWORD=123456
-p 5411:5432 postgres:latest
```

```
docker run -d --name lab12-worker1 --network lab12-net -e POSTGRES_PASSWORD=123456  
-p 5412:5432 postgres:latest
```

```
docker run -d --name lab12-worker2 --network lab12-net -e POSTGRES_PASSWORD=123456  
-p 5413:5432 postgres:latest
```

Verificamos que los tres contenedores estén activos:

```
docker ps
```

Finalmente, se inspecciona la red para comprobar que los tres servidores comparten la misma subred Docker:

```
docker network inspect lab12-net
```

Configuración del Worker 1

El primer worker almacenará el fragmento asociado al diagnóstico **Obesidad**.

```
docker exec -it lab12-worker1 psql -U postgres
```

Primero se crea el usuario remoto y la base de datos:

```
CREATE ROLE remote_user LOGIN PASSWORD '123456';  
  
CREATE DATABASE lab12_db;  
  
GRANT CONNECT ON DATABASE lab12_db TO remote_user;
```

Luego se cambia de base de datos. Este comando debe ejecutarse individualmente dentro de **psql**.

```
\c lab12_db
```

Se crea el esquema remoto, la tabla física correspondiente al fragmento de Obesidad y los permisos para el usuario remoto:

```
CREATE SCHEMA IF NOT EXISTS lab12_remote AUTHORIZATION remote_user;
```

```
GRANT USAGE, CREATE ON SCHEMA lab12_remote TO remote_user;

CREATE TABLE IF NOT EXISTS lab12_remote.atencionmedica_obesidad (
    dni CHAR(8),
    codmedico INTEGER NOT NULL,
    ciudad VARCHAR(50) NOT NULL,
    diagnostico VARCHAR(50) NOT NULL,
    peso DECIMAL(5,2) NOT NULL,
    talla DECIMAL(4,2) NOT NULL,
    presionarterial VARCHAR(10) NOT NULL,
    edad INTEGER NOT NULL CHECK (edad >= 0),
    fechaatencion DATE NOT NULL
);

GRANT SELECT, INSERT, UPDATE, DELETE
ON TABLE lab12_remote.atencionmedica_obesidad
TO remote_user;
```

Configuración del Worker 2

El segundo worker almacenará el fragmento asociado al diagnóstico **Hipertensión**.

```
docker exec -it lab12-worker2 psql -U postgres
```

Se crea el usuario remoto y la base de datos:

```
CREATE ROLE remote_user LOGIN PASSWORD '123456';

CREATE DATABASE lab12_db;

GRANT CONNECT ON DATABASE lab12_db TO remote_user;
```

Se cambia de base de datos de manera individual:

```
\c lab12_db
```

Luego se crea el esquema, la tabla física y los permisos:

```
CREATE SCHEMA IF NOT EXISTS lab12_remote AUTHORIZATION remote_user;

GRANT USAGE, CREATE ON SCHEMA lab12_remote TO remote_user;

CREATE TABLE IF NOT EXISTS lab12_remote.atencionmedica_hipertension (
    dni CHAR(8),
    codmedico INTEGER NOT NULL,
```

```
ciudad VARCHAR(50) NOT NULL,  
diagnostico VARCHAR(50) NOT NULL,  
peso DECIMAL(5,2) NOT NULL,  
talla DECIMAL(4,2) NOT NULL,  
presionarterial VARCHAR(10) NOT NULL,  
edad INTEGER NOT NULL CHECK (edad >= 0),  
fechaatencion DATE NOT NULL  
);  
  
GRANT SELECT, INSERT, UPDATE, DELETE  
ON TABLE lab12_remote.atencionmedica_hipertension  
TO remote_user;
```

Configuración del servidor master

El servidor master será el coordinador de la arquitectura distribuida.

```
docker exec -it lab12-master psql -U postgres
```

Se crea la base de datos:

```
CREATE DATABASE lab12_db;
```

Luego se cambia de base:

```
\c lab12_db
```

Se habilitan las extensiones necesarias y se crea el esquema del master:

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;  
  
CREATE EXTENSION IF NOT EXISTS dblink;  
  
CREATE SCHEMA lab12_master;
```

Conexión del master hacia los workers

Se definen los servidores foráneos. Dentro de Docker se utilizan los nombres de los contenedores como host, no `localhost`.

```
CREATE SERVER worker1_server  
FOREIGN DATA WRAPPER postgres_fdw
```



```
OPTIONS (  
    host 'lab12-worker1',  
    dbname 'lab12_db',  
    port '5432'  
);
```

```
CREATE SERVER worker2_server  
FOREIGN DATA WRAPPER postgres_fdw  
OPTIONS (  
    host 'lab12-worker2',  
    dbname 'lab12_db',  
    port '5432'  
);
```

Se crean los mapeos de usuario para autenticar al servidor master frente a los workers:

```
CREATE USER MAPPING FOR CURRENT_USER  
SERVER worker1_server  
OPTIONS (  
    user 'remote_user',  
    password '123456'  
);
```

```
CREATE USER MAPPING FOR CURRENT_USER  
SERVER worker2_server  
OPTIONS (  
    user 'remote_user',  
    password '123456'  
);
```

Verificamos los servidores foráneos creados:

```
SELECT srvname, srvoptions  
FROM pg_foreign_server  
WHERE srvname IN ('worker1_server', 'worker2_server');
```

```
lab12_db=# SELECT srvname, srvoptions
FROM pg_foreign_server
WHERE srvname IN ('worker1_server', 'worker2_server');
   srvname   |          srvoptions
-----+-----
worker1_server | {host=lab12-worker1,dbname=lab12_db,port=5432}
worker2_server | {host=lab12-worker2,dbname=lab12_db,port=5432}
(2 rows)
```

Creación de la tabla particionada distribuida

En el master se crea la tabla principal particionada por el atributo **Diagnostico**.

```
CREATE TABLE lab12_master.atencionmedica (
  dni CHAR(8),
  codmedico INTEGER NOT NULL,
  ciudad VARCHAR(50) NOT NULL,
  diagnostico VARCHAR(50) NOT NULL,
  peso DECIMAL(5,2) NOT NULL,
  talla DECIMAL(4,2) NOT NULL,
  presionarterial VARCHAR(10) NOT NULL,
  edad INTEGER NOT NULL CHECK (edad >= 0),
  fechaatencion DATE NOT NULL
) PARTITION BY LIST (diagnostico);
```

Se crean dos particiones locales en el master:

```
CREATE TABLE lab12_master.atencionmedica_diabetes
PARTITION OF lab12_master.atencionmedica
FOR VALUES IN ('Diabetes');
```

```
CREATE TABLE lab12_master.atencionmedica_cardiopatía
PARTITION OF lab12_master.atencionmedica
FOR VALUES IN ('Cardiopatía');
```

Se crean dos particiones foráneas. Aunque se registran en el master, sus datos físicos se almacenan en los workers.

```
CREATE FOREIGN TABLE lab12_master.atencionmedica_obesidad
PARTITION OF lab12_master.atencionmedica
FOR VALUES IN ('Obesidad')
SERVER worker1_server
OPTIONS (
  schema_name 'lab12_remote',
```

```

        table_name 'atencionmedica_obesidad'
    );

```

```

CREATE FOREIGN TABLE lab12_master.atencionmedica_hipertension
PARTITION OF lab12_master.atencionmedica
FOR VALUES IN ('Hipertensión')
SERVER worker2_server
OPTIONS (
    schema_name 'lab12_remote',
    table_name 'atencionmedica_hipertension'
);

```

La distribución inicial queda definida de la siguiente manera:

```

lab12-master
├── Diabetes
└── Cardiopatía

lab12-worker1
└── Obesidad

lab12-worker2
└── Hipertensión

```

Poblamiento de la tabla distribuida

Se insertan 60 000 registros sintéticos desde la tabla padre. PostgreSQL redirige cada registro automáticamente hacia su fragmento local o remoto según el diagnóstico.

```

INSERT INTO lab12_master.atencionmedica
(
    dni, codmedico, ciudad, diagnostico, peso, talla,
    presionarterial, edad, fechaatencion
)
SELECT
    lpad(g::text, 8, '0') AS dni,
    floor(random() * 200 + 1)::int AS codmedico,
    (
        ARRAY[
            'Lima', 'Callao', 'Arequipa', 'Cusco', 'Trujillo', 'Piura',
            'Chiclayo', 'Iquitos', 'Huancayo', 'Tacna', 'Pucallpa',
            'Juliaca', 'Ica', 'Ayacucho', 'Huaraz', 'Tumbes',
            'Moquegua', 'Chimbote'
        ]
    )[floor(random() * 18)::int + 1] AS ciudad,
    (
        ARRAY['Diabetes', 'Obesidad', 'Cardiopatía', 'Hipertensión']
    )

```

```

)[floor(random() * 4)::int + 1] AS diagnostico,
round((random() * 60 + 40)::numeric, 2) AS peso,
round((random() * 0.60 + 1.40)::numeric, 2) AS talla,
floor(random() * 60 + 90)::int || '/' ||
floor(random() * 40 + 60)::int AS presionarterial,
floor(random() * 96)::int AS edad,
DATE '2024-01-01' + floor(random() * 730)::int AS fechaatencion
FROM generate_series(1, 60000) AS g;

```

Verificamos que los registros hayan sido distribuidos entre las cuatro particiones:

```

SELECT
  tableoid::regclass AS particion,
  diagnostico,
  COUNT(*) AS registros
FROM lab12_master.atencionmedica
GROUP BY tableoid::regclass, diagnostico
ORDER BY particion;

```

```

lab12_db=# SELECT
  tableoid::regclass AS particion,
  diagnostico,
  COUNT(*) AS registros
FROM lab12_master.atencionmedica
GROUP BY tableoid::regclass, diagnostico
ORDER BY particion;

```

particion	diagnostico	registros
lab12_master.atencionmedica_diabetes	Diabetes	15029
lab12_master.atencionmedica_cardiopatía	Cardiopatía	15047
lab12_master.atencionmedica_obesidad	Obesidad	15093
lab12_master.atencionmedica_hipertension	Hipertensión	14831

(4 rows)

Catálogo de fragmentos

Se crea un catálogo que registra la ubicación de cada diagnóstico. Este catálogo será utilizado posteriormente por el procedimiento dinámico distribuido.

```

CREATE TABLE lab12_master.catalogo_fragmentos (
  diagnostico VARCHAR(50) PRIMARY KEY,
  ubicacion VARCHAR(10) NOT NULL,
  servidor VARCHAR(30),
  tabla_particion VARCHAR(80) NOT NULL
);

```

```
INSERT INTO lab12_master.catalogo_fragmentos
(diagnostico, ubicacion, servidor, tabla_particion)
VALUES
('Diabetes', 'LOCAL', NULL, 'atencionmedica_diabetes'),
('Cardiopatía', 'LOCAL', NULL, 'atencionmedica_cardiopatia'),
('Obesidad', 'REMOTO', 'worker1_server', 'atencionmedica_obesidad'),
('Hipertensión', 'REMOTO', 'worker2_server', 'atencionmedica_hipertension');
```

Función para normalizar diagnósticos

La función transforma un diagnóstico en un nombre válido para una tabla física. Por ejemplo, **Insuficiencia Renal** se transforma en **insuficiencia_renal**.

```
CREATE OR REPLACE FUNCTION lab12_master.normalizar_particion(p_texto TEXT)
RETURNS TEXT
LANGUAGE sql
IMMUTABLE
AS $$
    SELECT trim(
        both '_' FROM
        regexp_replace(
            translate(lower($1), 'áéíóúüñ', 'aeiouun'),
            '^[^a-z0-9]+',
            '_',
            'g'
        )
    );
$$;
```

Procedimiento dinámico distribuido

El procedimiento verifica si existe un fragmento para el diagnóstico recibido. Si no existe, selecciona el worker con menor cantidad de fragmentos, crea físicamente la tabla remota mediante **dblink_exec**, registra una nueva tabla foránea en el master y finalmente inserta el registro.

```
CREATE OR REPLACE PROCEDURE lab12_master.insertar_atencion_distribuida(
    p_dni CHAR(8),
    p_codmedico INTEGER,
    p_ciudad VARCHAR(50),
    p_diagnostico VARCHAR(50),
    p_peso DECIMAL(5,2),
    p_talla DECIMAL(4,2),
    p_presionarterial VARCHAR(10),
    p_edad INTEGER,
    p_fechaatencion DATE
)
LANGUAGE plpgsql
```

```
AS $$
DECLARE
    v_existe BOOLEAN;
    v_servidor TEXT;
    v_conexion TEXT;
    v_tabla TEXT;
BEGIN
    SELECT EXISTS (
        SELECT 1
        FROM lab12_master.catalogo_fragmentos
        WHERE diagnostico = p_diagnostico
    )
    INTO v_existe;

    IF NOT v_existe THEN
        SELECT CASE
            WHEN COUNT(*) FILTER (
                WHERE servidor = 'worker1_server'
            ) <= COUNT(*) FILTER (
                WHERE servidor = 'worker2_server'
            )
            THEN 'worker1_server'
            ELSE 'worker2_server'
        END
        INTO v_servidor
        FROM lab12_master.catalogo_fragmentos
        WHERE ubicacion = 'REMOTO';

        v_tabla := 'atencionmedica_' ||
            lab12_master.normalizar_particion(p_diagnostico);

        IF v_servidor = 'worker1_server' THEN
            v_conexion :=
                'host=lab12-worker1 port=5432 dbname=lab12_db user=remote_user
password=123456';
        ELSE
            v_conexion :=
                'host=lab12-worker2 port=5432 dbname=lab12_db user=remote_user
password=123456';
        END IF;

        PERFORM dblink_exec(
            v_conexion,
            format(
                'CREATE TABLE IF NOT EXISTS lab12_remote.%I (
                    dni CHAR(8),
                    codmedico INTEGER NOT NULL,
                    ciudad VARCHAR(50) NOT NULL,
                    diagnostico VARCHAR(50) NOT NULL,
                    peso DECIMAL(5,2) NOT NULL,
                    talla DECIMAL(4,2) NOT NULL,
                    presionarterial VARCHAR(10) NOT NULL,
                    edad INTEGER NOT NULL CHECK (edad >= 0),
                    fechaatencion DATE NOT NULL
                )'
            )
        );
    END IF;
END;
```

```

        ),
        v_tabla
    )
);

EXECUTE format(
    'CREATE FOREIGN TABLE lab12_master.%I
    PARTITION OF lab12_master.atencionmedica
    FOR VALUES IN (%L)
    SERVER %I
    OPTIONS (schema_name %L, table_name %L)',
    v_tabla,
    p_diagnostico,
    v_servidor,
    'lab12_remote',
    v_tabla
);

INSERT INTO lab12_master.catalogo_fragmentos
VALUES (
    p_diagnostico,
    'REMOTO',
    v_servidor,
    v_tabla
);

RAISE NOTICE 'Fragmento % creado en %',
    v_tabla, v_servidor;
END IF;

INSERT INTO lab12_master.atencionmedica
VALUES (
    p_dni, p_codmedico, p_ciudad, p_diagnostico,
    p_peso, p_talla, p_presionarterial,
    p_edad, p_fechaatencion
);
END;
$$;

```

Prueba de creación dinámica distribuida

Se insertan tres diagnósticos que no pertenecen a los fragmentos iniciales. El procedimiento debe crear las particiones correspondientes en los servidores remotos.

```

CALL lab12_master.insertar_atencion_distribuida(
    '91000001', 401, 'Lima', 'Asma',
    63.00, 1.63, '118/76', 28, DATE '2025-04-01'
);

```

```
CALL lab12_master.insertar_atencion_distribuida(
  '91000002', 402, 'Cusco', 'Anemia',
  58.00, 1.59, '110/70', 24, DATE '2025-04-02'
);
```

```
CALL lab12_master.insertar_atencion_distribuida(
  '91000003', 403, 'Piura', 'Gastritis',
  71.00, 1.70, '120/80', 35, DATE '2025-04-03'
);
```

```
CREATE PROCEDURE
lab12_db=# CALL lab12_master.insertar_atencion_distribuida(
  '91000001', 401, 'Lima', 'Asma',
  63.00, 1.63, '118/76', 28, DATE '2025-04-01'
);
CALL lab12_master.insertar_atencion_distribuida(
  '91000002', 402, 'Cusco', 'Anemia',
  58.00, 1.59, '110/70', 24, DATE '2025-04-02'
);
CALL lab12_master.insertar_atencion_distribuida(
  '91000003', 403, 'Piura', 'Gastritis',
  71.00, 1.70, '120/80', 35, DATE '2025-04-03'
);
NOTICE: Fragmento atencionmedica_asma creado en worker1_server
CALL
NOTICE: Fragmento atencionmedica_anemia creado en worker2_server
CALL
NOTICE: Fragmento atencionmedica_gastritis creado en worker1_server
CALL
```

Verificamos la asignación de los nuevos fragmentos:

```
SELECT *
FROM lab12_master.catalogo_fragmentos
ORDER BY ubicacion, servidor, diagnostico;
```



```
lab12_db=# SELECT *
FROM lab12_master.catalogo_fragmentos
ORDER BY ubicacion, servidor, diagnostico;
```

diagnostico	ubicacion	servidor	tabla_particion
Cardiopatía	LOCAL		atencionmedica_cardiopatia
Diabetes	LOCAL		atencionmedica_diabetes
Asma	REMOTO	worker1_server	atencionmedica_asma
Gastritis	REMOTO	worker1_server	atencionmedica_gastritis
Obesidad	REMOTO	worker1_server	atencionmedica_obesidad
Anemia	REMOTO	worker2_server	atencionmedica_anemia
Hipertensión	REMOTO	worker2_server	atencionmedica_hipertension

(7 rows)

Verificamos en qué partición se almacenó cada nuevo registro:

```
SELECT
    tableoid::regclass AS particion,
    diagnostico,
    COUNT(*) AS registros
FROM lab12_master.atencionmedica
WHERE dni IN ('91000001', '91000002', '91000003')
GROUP BY tableoid::regclass, diagnostico
ORDER BY particion;
```

```
lab12_db=# SELECT
    tableoid::regclass AS particion,
    diagnostico,
    COUNT(*) AS registros
FROM lab12_master.atencionmedica
WHERE dni IN ('91000001', '91000002', '91000003')
GROUP BY tableoid::regclass, diagnostico
ORDER BY particion;
```

particion	diagnostico	registros
lab12_master.atencionmedica_asma	Asma	1
lab12_master.atencionmedica_anemia	Anemia	1
lab12_master.atencionmedica_gastritis	Gastritis	1

(3 rows)

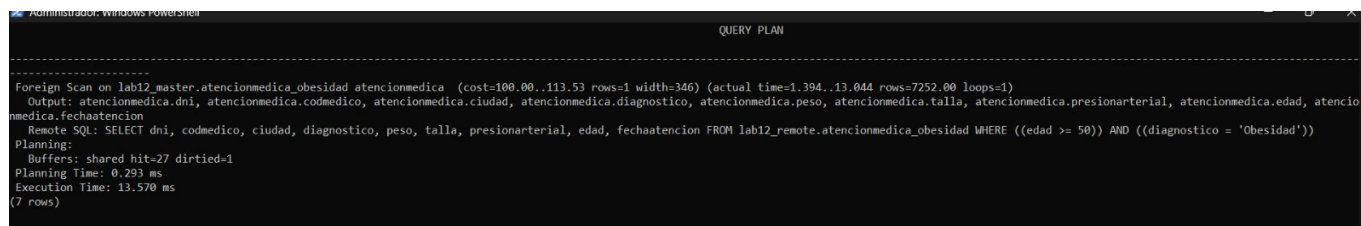
```
lab12_db=#
```

Consulta distribuida con postgres_fdw

La siguiente consulta accede al fragmento de Obesidad, alojado físicamente en `lab12-worker1`.

```
EXPLAIN (ANALYZE, VERBOSE, COSTS, BUFFERS)
SELECT *
```

```
FROM lab12_master.atencionmedica
WHERE diagnostico = 'Obesidad'
AND edad >= 50;
```



El plan muestra un operador Foreign Scan sobre lab12_master.atencionmedica_obesidad. Esto confirma que el master consultó el fragmento de Obesidad almacenado en lab12-worker1, en lugar de leer una tabla local.

La línea Remote SQL evidencia que los filtros diagnostico = 'Obesidad' y edad >= 50 fueron enviados al worker remoto. Por tanto, el filtrado se realiza en el servidor que contiene físicamente los datos, reduciendo la transferencia de registros hacia el master. La consulta retornó 7 252 filas y registró un tiempo de ejecución aproximado de 13.57 ms.

Consulta distribuida con dblink

Se abre una conexión persistente desde el master hacia el segundo worker.

```
SELECT dblink_connect(
    'conexion_worker2',
    'host=lab12-worker2 port=5432 dbname=lab12_db user=remote_user
password=123456'
);
```

Se ejecuta una consulta directa sobre el fragmento remoto de Hipertensión:

```
EXPLAIN (ANALYZE, VERBOSE, COSTS, BUFFERS)
SELECT *
FROM dblink(
    'conexion_worker2',
    'SELECT diagnostico,
        COUNT(*)::bigint AS total_atenciones,
        ROUND(AVG(edad), 2)::numeric AS promedio_edad
    FROM lab12_remote.atencionmedica_hipertension
    GROUP BY diagnostico'
) AS t(
    diagnostico VARCHAR(50),
    total_atenciones BIGINT,
    promedio_edad NUMERIC
);
```

```

QUERY PLAN
-----
Function Scan on public.dblink t (cost=0.00..10.00 rows=1000 width=158) (actual time=3.028..3.030 rows=1.00 loops=1)
  Output: diagnostico, total_atenciones, promedio_edad
  Function Call: dblink('conexion_worker2'::text, 'SELECT diagnostico,
    COUNT(*)::bigint AS total_atenciones,
    ROUND(AVG(edad), 2)::numeric AS promedio_edad
  FROM lab12_remote.atencionmedica_hipertension
  GROUP BY diagnostico'::text)
Planning Time: 0.024 ms
Execution Time: 3.122 ms
(9 rows)

```

El plan presenta el operador Function Scan on public.dblink, confirmando que el servidor master ejecutó una consulta directa sobre lab12-worker2 mediante la extensión dblink.

La consulta remota agrupa las atenciones de Hipertensión y calcula la cantidad de registros junto con la edad promedio. A diferencia de postgres_fdw, dblink se invoca como una función y no como una tabla foránea integrada en el plan de ejecución. El tiempo de ejecución fue aproximadamente 3.12 ms.

Finalmente, se cierra la conexión:

```
SELECT dblink_disconnect('conexion_worker2');
```

Consulta integradora de los tres servidores

La siguiente consulta consolida los resultados de las dos particiones locales y las particiones foráneas.

```

EXPLAIN (ANALYZE, VERBOSE, COSTS, BUFFERS)
SELECT
  diagnostico,
  COUNT(*) AS total_atenciones,
  ROUND(AVG(edad), 2) AS promedio_edad
FROM lab12_master.atencionmedica
GROUP BY diagnostico
ORDER BY diagnostico;

```

```

QUERY PLAN
-----
Sort (cost=2241.52..2242.02 rows=200 width=60) (actual time=35.792..35.796 rows=7.00 loops=1)
  Output: atencionmedica.diagnostico, (count(*)), (round(avg(atencionmedica.edad), 2))
  Sort Key: atencionmedica.diagnostico
  Sort Method: quicksort Memory: 25kB
  Buffers: shared hit=352
-> HashAggregate (cost=2230.87..2233.87 rows=200 width=60) (actual time=35.774..35.779 rows=7.00 loops=1)
  Output: atencionmedica.diagnostico, count(*), round(avg(atencionmedica.edad), 2)
  Group Key: atencionmedica.diagnostico
  Batches: 1 Memory Usage: 32kB
  Buffers: shared hit=352
-> Append (cost=100.00..1984.27 rows=32881 width=24) (actual time=0.270..30.450 rows=60003.00 loops=1)
  Buffers: shared hit=352
-> Foreign Scan on lab12_master.atencionmedica_anemia atencionmedica_1 (cost=100.00..233.42 rows=561 width=122) (actual time=0.269..0.270 rows=1.00 loops=1)
  Output: atencionmedica_1.diagnostico, atencionmedica_1.edad
  Remote SQL: SELECT diagnostico, edad FROM lab12_remote.atencionmedica_anemia
-> Foreign Scan on lab12_master.atencionmedica_asma atencionmedica_2 (cost=100.00..233.42 rows=561 width=122) (actual time=0.292..0.293 rows=1.00 loops=1)
  Output: atencionmedica_2.diagnostico, atencionmedica_2.edad
  Remote SQL: SELECT diagnostico, edad FROM lab12_remote.atencionmedica_asma
-> Seq Scan on lab12_master.atencionmedica_cardiopatia atencionmedica_3 (cost=0.00..331.47 rows=15047 width=17) (actual time=0.007..1.042 rows=15047.00 loops=1)
  Output: atencionmedica_3.diagnostico, atencionmedica_3.edad
  Buffers: shared hit=181
-> Seq Scan on lab12_master.atencionmedica_diabetes atencionmedica_4 (cost=0.00..321.29 rows=15029 width=13) (actual time=0.013..0.915 rows=15029.00 loops=1)
  Output: atencionmedica_4.diagnostico, atencionmedica_4.edad
  Buffers: shared hit=171
-> Foreign Scan on lab12_master.atencionmedica_gastritis atencionmedica_5 (cost=100.00..233.42 rows=561 width=122) (actual time=0.367..0.367 rows=1.00 loops=1)
  Output: atencionmedica_5.diagnostico, atencionmedica_5.edad
  Remote SQL: SELECT diagnostico, edad FROM lab12_remote.atencionmedica_gastritis
-> Foreign Scan on lab12_master.atencionmedica_hipertension atencionmedica_6 (cost=100.00..233.42 rows=561 width=122) (actual time=0.483..12.663 rows=14831.00 loops=1)
  Output: atencionmedica_6.diagnostico, atencionmedica_6.edad
  Remote SQL: SELECT diagnostico, edad FROM lab12_remote.atencionmedica_hipertension
-> Foreign Scan on lab12_master.atencionmedica_obesidad atencionmedica_7 (cost=100.00..233.42 rows=561 width=122) (actual time=0.241..12.507 rows=15093.00 loops=1)
  Output: atencionmedica_7.diagnostico, atencionmedica_7.edad
  Remote SQL: SELECT diagnostico, edad FROM lab12_remote.atencionmedica_obesidad
Planning:
  Buffers: shared hit=117 dirtied=6
Planning Time: 0.352 ms
Execution Time: 36.506 ms
(37 rows)

```

El nodo Append integra los resultados provenientes de todas las particiones. Los operadores Seq Scan corresponden a las particiones locales de Diabetes y Cardiopatía, ubicadas en el servidor master.

Los operadores Foreign Scan corresponden a los fragmentos almacenados en los workers: Obesidad, Hipertensión y los diagnósticos creados dinámicamente, como Asma, Anemia y Gastritis. Las líneas Remote SQL muestran las consultas enviadas a los servidores remotos.

Finalmente, el master aplica HashAggregate para agrupar por diagnóstico y calcular la cantidad de atenciones y la edad promedio. Después utiliza Sort para ordenar el resultado. La consulta integró 60 003 registros y tuvo un tiempo de ejecución aproximado de 36.51 ms.

Conclusión de P3

La implementación permitió distribuir los fragmentos de AtencionMedica entre un servidor coordinador y dos servidores remotos. postgres_fdw permitió integrar los fragmentos remotos como particiones de la tabla principal, mientras que dblink se utilizó para crear dinámicamente tablas físicas en los workers y ejecutar consultas directas sobre ellos.

Los planes de ejecución demostraron que PostgreSQL diferencia las particiones locales mediante Seq Scan y las particiones remotas mediante Foreign Scan. De esta forma, se comprobó la ejecución de consultas distribuidas y la asignación dinámica de nuevos diagnósticos entre los workers.

P4. Algoritmos distribuidos localmente

Primero creamos la tabla Pacientes, con la partition por range de 'CiudadOrigen':

```
CREATE TABLE Pacientes (  
  DNI CHAR(8),  
  Nombre VARCHAR(50) NOT NULL,  
  Apellidos VARCHAR(100) NOT NULL,  
  FechaNacimiento DATE NOT NULL,  
  Sexo CHAR(1) NOT NULL CHECK (Sexo IN ('M', 'F')),  
  CiudadOrigen VARCHAR(50) NOT NULL,  
  PRIMARY KEY (DNI, CiudadOrigen)  
) PARTITION BY RANGE (CiudadOrigen);
```

```
Query Query History
1 CREATE TABLE Pacientes (
2     DNI CHAR(8),
3     Nombre VARCHAR(50) NOT NULL,
4     Apellidos VARCHAR(100) NOT NULL,
5     FechaNacimiento DATE NOT NULL,
6     Sexo CHAR(1) NOT NULL CHECK (Sexo IN ('M', 'F')),
7     CiudadOrigen VARCHAR(50) NOT NULL,
8     PRIMARY KEY (DNI, CiudadOrigen)
9 ) PARTITION BY RANGE (CiudadOrigen);

Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 70 msec.
```

Creamos las particiones por rango del vector ["H", "P"]

```
CREATE TABLE Pacientes_F1 PARTITION OF Pacientes
FOR VALUES FROM (MINVALUE) TO ('H');

CREATE TABLE Pacientes_F2 PARTITION OF Pacientes
FOR VALUES FROM ('H') TO ('P');

CREATE TABLE Pacientes_F3 PARTITION OF Pacientes
FOR VALUES FROM ('P') TO (MAXVALUE);
```

```
Query Query History
1 CREATE TABLE Pacientes_F1 PARTITION OF Pacientes
2     FOR VALUES FROM (MINVALUE) TO ('H');
3
4 CREATE TABLE Pacientes_F2 PARTITION OF Pacientes
5     FOR VALUES FROM ('H') TO ('P');
6
7 CREATE TABLE Pacientes_F3 PARTITION OF Pacientes
8     FOR VALUES FROM ('P') TO (MAXVALUE);

Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 109 msec.
```

El enunciado dice DNI CHAR(8) PRIMARY KEY, pero PostgreSQL exige que toda PK/UNIQUE en una tabla particionada incluya las columnas de partición. Por eso uso PRIMARY KEY (DNI, CiudadOrigen).

Ahora poblamos la tabla con registros sinteticos:

```
INSERT INTO Pacientes (DNI, Nombre, Apellidos, FechaNacimiento, Sexo,
CiudadOrigen)
SELECT
    LPAD(g::text, 8, '0') AS DNI,
    (ARRAY['Juan','Maria','Carlos','Ana','Luis','Sofia','Pedro',
        'Lucia','Diego','Camila','Jorge','Valeria','Andres',
        'Daniela','Miguel','Fernanda','Ricardo','Patricia',
        'Roberto','Gabriela'])[(random()*19+1)::int] AS Nombre,
    (ARRAY['Garcia','Rodriguez','Lopez','Martinez','Perez',
        'Gonzalez','Sanchez','Ramirez','Torres','Flores',
        'Rivera','Vargas','Castillo','Romero','Morales',
        'Ortiz','Gutierrez','Chavez','Mendoza','Aguilar'])
        [(random()*19+1)::int] AS Apellidos,
    DATE '1940-01-01' + ((random()*30000)::int) AS
FechaNacimiento,
    (ARRAY['M','F'])[(random()*1+1)::int] AS Sexo,
    (ARRAY['Lima','Callao','Arequipa','Cusco','Trujillo','Piura',
        'Chiclayo','Iquitos','Huancayo','Tacna','Pucallpa',
        'Juliaca','Ica','Ayacucho','Huaraz','Tumbes',
        'Moquegua','Chimbote'])[(random()*17+1)::int] AS
CiudadOrigen
FROM generate_series(1, 60000) g;
```

Query	Query History
1	INSERT INTO Pacientes (DNI, Nombre, Apellidos, FechaNacimiento, Sexo, CiudadOrigen)
2	SELECT
3	LPAD(g::text, 8, '0') AS DNI,
4	(ARRAY['Juan','Maria','Carlos','Ana','Luis','Sofia','Pedro',
5	'Lucia','Diego','Camila','Jorge','Valeria','Andres',
6	'Daniela','Miguel','Fernanda','Ricardo','Patricia',
7	'Roberto','Gabriela'])[(random()*19+1)::int] AS Nombre,
8	(ARRAY['Garcia','Rodriguez','Lopez','Martinez','Perez',
9	'Gonzalez','Sanchez','Ramirez','Torres','Flores',
10	'Rivera','Vargas','Castillo','Romero','Morales',
11	'Ortiz','Gutierrez','Chavez','Mendoza','Aguilar'])
12	[(random()*19+1)::int] AS Apellidos,
13	DATE '1940-01-01' + ((random()*30000)::int) AS FechaNacimiento,
14	(ARRAY['M','F'])[(random()*1+1)::int] AS Sexo,
15	(ARRAY['Lima','Callao','Arequipa','Cusco','Trujillo','Piura',
16	'Chiclayo','Iquitos','Huancayo','Tacna','Pucallpa',
17	'Juliaca','Ica','Ayacucho','Huaraz','Tumbes',
18	'Moquegua','Chimbote'])[(random()*17+1)::int] AS CiudadOrigen
19	FROM generate_series(1, 60000) g;

Data Output	Messages	Notifications
INSERT 0 60000		
Query returned successfully in 248 msec.		

Q1. SELECT * FROM Pacientes ORDER BY FechaNacimiento

```
-- R1: filas de Pacientes_F1 ordenadas localmente por FechaNacimiento
CREATE TEMP TABLE r1 ON COMMIT DROP AS
SELECT * FROM Pacientes_F1 ORDER BY FechaNacimiento;

-- R2: filas de Pacientes_F2 ordenadas localmente
CREATE TEMP TABLE r2 ON COMMIT DROP AS
SELECT * FROM Pacientes_F2 ORDER BY FechaNacimiento;

-- R3: filas de Pacientes_F3 ordenadas localment
CREATE TEMP TABLE r3 ON COMMIT DROP AS
SELECT * FROM Pacientes_F3 ORDER BY FechaNacimiento;

-- Merge de R1, R2, R3 en el master
EXPLAIN ANALYZE
SELECT * FROM (
    SELECT * FROM r1
    UNION ALL SELECT * FROM r2
    UNION ALL SELECT * FROM r3
) m
ORDER BY FechaNacimiento;
```


Query

Query History

```

1  -- R1: filas de Pacientes_F1 ordenadas localmente por FechaNacimiento
2  CREATE TEMP TABLE r1 ON COMMIT DROP AS
3  SELECT * FROM Pacientes_F1 ORDER BY FechaNacimiento;
4
5  -- R2: filas de Pacientes_F2 ordenadas localmente
6  CREATE TEMP TABLE r2 ON COMMIT DROP AS
7  SELECT * FROM Pacientes_F2 ORDER BY FechaNacimiento;
8
9  -- R3: filas de Pacientes_F3 ordenadas localment
10 CREATE TEMP TABLE r3 ON COMMIT DROP AS
11 SELECT * FROM Pacientes_F3 ORDER BY FechaNacimiento;
12
13 -- Merge de R1, R2, R3 en el master
14 EXPLAIN ANALYZE
15 SELECT * FROM (
16     SELECT * FROM r1
17     UNION ALL SELECT * FROM r2
18     UNION ALL SELECT * FROM r3
19 ) m
20 ORDER BY FechaNacimiento;

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🔄

📥

📈

SQL

	QUERY PLAN	
	text	🔒
1	Sort (cost=1132.03..1151.38 rows=7740 width=502) (actual time=10.356..13.041 rows=60000 loops=1)	
2	Sort Key: r1.fechanacimiento	
3	Sort Method: external merge Disk: 2864kB	
4	-> Append (cost=0.00..632.10 rows=7740 width=502) (actual time=0.004..3.826 rows=60000 loops=1)	
5	-> Seq Scan on r1 (cost=0.00..190.90 rows=2490 width=502) (actual time=0.003..0.718 rows=19245 loo...	
6	-> Seq Scan on r2 (cost=0.00..226.55 rows=2955 width=502) (actual time=0.002..0.784 rows=22910 loo...	
7	-> Seq Scan on r3 (cost=0.00..175.95 rows=2295 width=502) (actual time=0.003..0.645 rows=17845 loo...	
8	Planning Time: 0.069 ms	
9	Execution Time: 14.177 ms	

Q2. SELECT DISTINCT CiudadOrigen FROM Pacientes

```

-- R1: ciudades distintas en F1 < 'H'
CREATE TEMP TABLE r1 ON COMMIT DROP AS
SELECT DISTINCT CiudadOrigen FROM Pacientes_F1;

-- R2: ciudades distintas en F2 'H'<..< 'P'
CREATE TEMP TABLE r2 ON COMMIT DROP AS
SELECT DISTINCT CiudadOrigen FROM Pacientes_F2;

-- R3: ciudades distintas en F3 'P'>
CREATE TEMP TABLE r3 ON COMMIT DROP AS

```



```

SELECT DISTINCT CiudadOrigen FROM Pacientes_F3;

-- Union All
EXPLAIN ANALYZE
SELECT CiudadOrigen FROM r1
UNION ALL SELECT CiudadOrigen FROM r2
UNION ALL SELECT CiudadOrigen FROM r3;

```

Query

Query History

```

1  -- R1: ciudades distintas en F1 < 'H'
2  CREATE TEMP TABLE r1 ON COMMIT DROP AS
3  SELECT DISTINCT CiudadOrigen FROM Pacientes_F1;
4
5  -- R2: ciudades distintas en F2 'H'<..< 'P'
6  CREATE TEMP TABLE r2 ON COMMIT DROP AS
7  SELECT DISTINCT CiudadOrigen FROM Pacientes_F2;
8
9  -- R3: ciudades distintas en F3 'P'>
10 CREATE TEMP TABLE r3 ON COMMIT DROP AS
11 SELECT DISTINCT CiudadOrigen FROM Pacientes_F3;
12
13 -- Union All
14 EXPLAIN ANALYZE
15 SELECT CiudadOrigen FROM r1
16 UNION ALL SELECT CiudadOrigen FROM r2
17 UNION ALL SELECT CiudadOrigen FROM r3;

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	QUERY PLAN	
	text	🔒
1	Append (cost=0.00..54.75 rows=1650 width=118) (actual time=0.002..0.006 rows=18 loops=1)	
2	-> Seq Scan on r1 (cost=0.00..15.50 rows=550 width=118) (actual time=0.002..0.003 rows=6 loop...	
3	-> Seq Scan on r2 (cost=0.00..15.50 rows=550 width=118) (actual time=0.001..0.001 rows=7 loop...	
4	-> Seq Scan on r3 (cost=0.00..15.50 rows=550 width=118) (actual time=0.001..0.001 rows=5 loop...	
5	Planning Time: 0.049 ms	
6	Execution Time: 0.012 ms	

Q3. SELECT Diagnostico, AVG(Edad) AS PromEdad FROM AtencionMedica GROUP BY Diagnostico

```

-- R1: prom. edad para Diabetes (sitio: F1) → 1 fila
CREATE TEMP TABLE r1 ON COMMIT DROP AS
SELECT Diagnostico, AVG(Edad) AS PromEdad
FROM AtencionMedica_Diabetes GROUP BY Diagnostico;

```

```
-- R2: prom. edad para Obesidad (sitio: F2) → 1 fila
CREATE TEMP TABLE r2 ON COMMIT DROP AS
SELECT Diagnostico, AVG(Edad) AS PromEdad
FROM AtencionMedica_Obesidad GROUP BY Diagnostico;

-- R3: prom. edad para Cardiopatía
CREATE TEMP TABLE r3 ON COMMIT DROP AS
SELECT Diagnostico, AVG(Edad) AS PromEdad
FROM AtencionMedica_Cardiopatía GROUP BY Diagnostico;

-- R4: prom. edad para Hipertensión
CREATE TEMP TABLE r4 ON COMMIT DROP AS
SELECT Diagnostico, AVG(Edad) AS PromEdad
FROM AtencionMedica_Hipertension GROUP BY Diagnostico;

-- UNION ALL (los grupos no se solapan entre fragmentos).
EXPLAIN ANALYZE
SELECT * FROM r1
UNION ALL SELECT * FROM r2
UNION ALL SELECT * FROM r3
UNION ALL SELECT * FROM r4;
```

Query

Query History

```

1  -- R1: prom. edad para Diabetes (sitio: F1) → 1 fila
2  CREATE TEMP TABLE r1 ON COMMIT DROP AS
3  SELECT Diagnostico, AVG(Edad) AS PromEdad
4  FROM AtencionMedica_Diabetes GROUP BY Diagnostico;
5
6  -- R2: prom. edad para Obesidad (sitio: F2) → 1 fila
7  CREATE TEMP TABLE r2 ON COMMIT DROP AS
8  SELECT Diagnostico, AVG(Edad) AS PromEdad
9  FROM AtencionMedica_Obesidad GROUP BY Diagnostico;
10
11 -- R3: prom. edad para Cardiopatía
12 CREATE TEMP TABLE r3 ON COMMIT DROP AS
13 SELECT Diagnostico, AVG(Edad) AS PromEdad
14 FROM AtencionMedica_Cardiopatia GROUP BY Diagnostico;
15
16 -- R4: prom. edad para Hipertensión
17 CREATE TEMP TABLE r4 ON COMMIT DROP AS
18 SELECT Diagnostico, AVG(Edad) AS PromEdad
19 FROM AtencionMedica_Hipertension GROUP BY Diagnostico;
20

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	QUERY PLAN	
	text	🔒
1	Append (cost=0.00..67.00 rows=1800 width=150) (actual time=0.002..0.005 rows=4 loops=1)	
2	-> Seq Scan on r1 (cost=0.00..14.50 rows=450 width=150) (actual time=0.002..0.002 rows=1 loop...	
3	-> Seq Scan on r2 (cost=0.00..14.50 rows=450 width=150) (actual time=0.001..0.001 rows=1 loop...	
4	-> Seq Scan on r3 (cost=0.00..14.50 rows=450 width=150) (actual time=0.000..0.001 rows=1 loop...	
5	-> Seq Scan on r4 (cost=0.00..14.50 rows=450 width=150) (actual time=0.000..0.001 rows=1 loop...	
6	Planning Time: 0.062 ms	
7	Execution Time: 0.011 ms	

Q4. SELECT * FROM Pacientes NATURAL JOIN AtencionMedica

```

-- Proyecció local de DNI en cada fragmento de AtencionMedica.
CREATE TEMP TABLE dni_a1 ON COMMIT DROP AS SELECT DISTINCT DNI FROM
AtencionMedica_Diabetes;
CREATE TEMP TABLE dni_a2 ON COMMIT DROP AS SELECT DISTINCT DNI FROM
AtencionMedica_Obesidad;
CREATE TEMP TABLE dni_a3 ON COMMIT DROP AS SELECT DISTINCT DNI FROM
AtencionMedica_Cardiopatia;
CREATE TEMP TABLE dni_a4 ON COMMIT DROP AS SELECT DISTINCT DNI FROM
AtencionMedica_Hipertension;

-- Unión de DNIs a enviar al lado de Pacientes
CREATE TEMP TABLE dni_atencion ON COMMIT DROP AS

```

```
SELECT DNI FROM dni_a1
UNION SELECT DNI FROM dni_a2
UNION SELECT DNI FROM dni_a3
UNION SELECT DNI FROM dni_a4;

-- Semi-join local en cada fragmento de Pacientes
CREATE TEMP TABLE p_filt_1 ON COMMIT DROP AS
SELECT p.* FROM Pacientes_F1 p WHERE p.DNI IN (SELECT DNI FROM dni_atencion);

CREATE TEMP TABLE p_filt_2 ON COMMIT DROP AS
SELECT p.* FROM Pacientes_F2 p WHERE p.DNI IN (SELECT DNI FROM dni_atencion);

CREATE TEMP TABLE p_filt_3 ON COMMIT DROP AS
SELECT p.* FROM Pacientes_F3 p WHERE p.DNI IN (SELECT DNI FROM dni_atencion);

-- Unión del lado izquierdo del join ya reducido
CREATE TEMP TABLE p_reduced ON COMMIT DROP AS
SELECT * FROM p_filt_1
UNION ALL SELECT * FROM p_filt_2
UNION ALL SELECT * FROM p_filt_3;

-- Join final en el coordinador con AtencionMedica completa
EXPLAIN ANALYZE
SELECT * FROM p_reduced NATURAL JOIN AtencionMedica;
```

Lab13/postgres@localhost

Query Query History

```

12 CREATE TEMP TABLE dni_atencion ON COMMIT DROP AS
13 SELECT DNI FROM dni_a1
14 UNION SELECT DNI FROM dni_a2
15 UNION SELECT DNI FROM dni_a3
16 UNION SELECT DNI FROM dni_a4;
17
18 -- Semi-join local en cada fragmento de Pacientes
19 CREATE TEMP TABLE p_filt_1 ON COMMIT DROP AS
20 SELECT p.* FROM Pacientes_F1 p WHERE p.DNI IN (SELECT DNI FROM dni_atencion);
21
22 CREATE TEMP TABLE p_filt_2 ON COMMIT DROP AS

```

Data Output Messages Notifications

Showing rows: 1 to 17

QUERY PLAN	
text	
1	Merge Join (cost=9243.36..29665.86 rows=1350000 width=550) (actual time=154.947..183.800 rows=60000 loops=1)
2	Merge Cond: (p_reduced.dni = atencionmedica.dni)
3	-> Sort (cost=618.05..629.30 rows=4500 width=502) (actual time=49.457..50.308 rows=34972 loops=1)
4	Sort Key: p_reduced.dni
5	Sort Method: quicksort Memory: 3721kB
6	-> Seq Scan on p_reduced (cost=0.00..345.00 rows=4500 width=502) (actual time=0.006..1.621 rows=34972 loops=1)
7	-> Materialize (cost=8625.30..8925.30 rows=60000 width=57) (actual time=105.461..117.291 rows=60000 loops=1)
8	-> Sort (cost=8625.30..8775.30 rows=60000 width=57) (actual time=105.437..113.799 rows=60000 loops=1)
9	Sort Key: atencionmedica.dni
10	Sort Method: external merge Disk: 4232kB
11	-> Append (cost=0.00..1606.00 rows=60000 width=57) (actual time=0.040..9.830 rows=60000 loops=1)
12	-> Seq Scan on atencionmedica_cardiopatia atencionmedica_1 (cost=0.00..439.53 rows=19953 width=59) (actual time=0.038..3.878 rows=19953 loops=1)
13	-> Seq Scan on atencionmedica_diabetes atencionmedica_2 (cost=0.00..213.60 rows=9960 width=55) (actual time=0.021..1.185 rows=9960 loops=1)
14	-> Seq Scan on atencionmedica_hipertension atencionmedica_3 (cost=0.00..224.96 rows=10096 width=60) (actual time=0.011..1.204 rows=10096 loops=1)
15	-> Seq Scan on atencionmedica_obesidad atencionmedica_4 (cost=0.00..427.91 rows=19991 width=55) (actual time=0.017..1.583 rows=19991 loops=1)
16	Planning Time: 0.088 ms
17	Execution Time: 185.498 ms

Total rows: 17 Query complete 00:00:00.298

Explicación de Resultados

Q1 — `SELECT * FROM Pacientes ORDER BY FechaNacimiento`

Plan:

- **Append** recorre las tres temp tables locales (**r1**, **r2**, **r3**) ya ordenadas en sus respectivos fragmentos: **19 245 + 22 910 + 17 845 = 60 000 filas**.
- En el coordinador se aplica un **Sort** final con **Sort Method: external merge** (**Disk: 2864 kB**), reflejando el k-way merge clásico del algoritmo distribuido.
- **Execution Time = 14.18 ms**.

Observación: el desbalance entre fragmentos (F2 tiene más filas porque concentra Lima, Ica, Iquitos, Juliaca, Huancayo, etc.) es esperado por la fragmentación lexicográfica.

Q2 — `SELECT DISTINCT CiudadOrigen FROM Pacientes`

Plan:

- **Append** sobre **r1** (6 filas), **r2** (7 filas), **r3** (5 filas) = **18 ciudades distintas**.

- **Seq Scan** simple en cada temp table; no requiere re-deduplicación porque la fragmentación por **CiudadOrigen** garantiza que cada ciudad vive en un único fragmento, entonces solo basta **UNION ALL**.
- **Execution Time = 0.012 ms.**

Esta consulta evidencia la mejor optimización posible: el atributo del **DISTINCT** coincide con la clave de partición, así que **no hay solapamiento entre fragmentos** y el coordinador solo concatena resultados parciales pequeños.

Q3 — **SELECT Diagnostico, AVG(Edad) GROUP BY Diagnostico**

Plan:

- **Append** sobre **r1, r2, r3, r4**, cada una con **exactamente 1 fila = 4 filas finales**.
- Cada **Seq Scan** opera sobre un fragmento que contiene **un único valor de Diagnostico**, el **AVG(Edad)** local ya es la respuesta final para ese grupo.
- **Execution Time = 0.011 ms.**

Aquí se aprovecha que **GROUP BY** coincide con la clave de partición, no se necesita re-agregar. Si la columna del **GROUP BY** no fuera la de partición, habría que mover **SUM(Edad)** y **COUNT(*)** al coordinador y calcular **AVG = SUM_total / COUNT_total**.

Q4 — **SELECT * FROM Pacientes NATURAL JOIN AtencionMedica**

Plan:

- La clave del join (**DNI**) no coincide con ninguna clave de partición de las dos tablas, por lo que se requiere el **semi-join** implementado en la query:
 1. Proyección local π_{DNI} en cada uno de los 4 fragmentos de **AtencionMedica**
 2. Unión deduplicada -> **dni_atencion** (DNIs únicos enviados al lado de Pacientes).
 3. Semi-join local en cada fragmento de **Pacientes**, filtra los pacientes que tienen al menos una atención registrada.
 4. Unión del lado izquierdo reducido (**p_reduced**) y join final contra **AtencionMedica**.
- El semi-join reduce el tráfico al coordinador: en lugar de enviar tuplas completas de Pacientes, primero se filtra por la lista de DNIs presentes en atenciones.

Comparado con un **NATURAL JOIN** ingenuo sobre las tablas particionadas completas, el algoritmo distribuido evidencia claramente las 4 fases (proyección, envío, semi-join, join final) y demuestra el principio de "mover poco dato, no relaciones enteras".

Conclusiones

- **Q2 y Q3** son los casos óptimos: la clave de la operación (**DISTINCT / GROUP BY**) coincide con la clave de partición.
- **Q1** requiere merge porque ningún fragmento conoce el orden global, pero al menos cada fragmento entrega sus filas ya ordenadas (sort paralelo).
- **Q4** es el caso más caro porque la clave del join (**DNI**) no está alineada con ninguna fragmentación, justificando la optimización por semi-join.
- Los tiempos de Q2 y Q3 son sub-milisegundo porque las temp tables intermedias son diminutas (18 y 4 filas respectivamente).

P5. Algoritmos distribuidos en tres servidores

En esta sección se replican las cuatro consultas distribuidas de P4, pero ahora los fragmentos residen físicamente en tres contenedores Docker independientes. La comunicación entre el servidor coordinador y los workers se implementa con la extensión `postgres_fdw`.

Arquitectura

Servidor	Contenedor	Puerto	Fragmentos alojados
Master (coordinador)	<code>pg_master</code>	5432	<code>AtencionMedica_Diabetes</code> , <code>Pacientes_F1</code>
Worker 1	<code>pg_worker1</code>	5433	<code>AtencionMedica_Obesidad</code> , <code>AtencionMedica_Cardiopatia</code> , <code>Pacientes_F2</code>
Worker 2	<code>pg_worker2</code>	5434	<code>AtencionMedica_Hipertension</code> , <code>Pacientes_F3</code>

La fragmentación de `Pacientes` respeta el vector `["H", "P"]`:

- **F1** (Master): `CiudadOrigen < 'H'` → Arequipa, Ayacucho, Callao, Chiclayo, Chimbote, Cusco
- **F2** (Worker1): `'H' ≤ CiudadOrigen < 'P'` → Huancayo, Huaraz, Ica, Iquitos, Juliaca, Lima, Moquegua
- **F3** (Worker2): `CiudadOrigen ≥ 'P'` → Piura, Pucallpa, Tacna, Trujillo, Tumbes

Entorno Docker y configuración FDW

Los tres contenedores se levantan con `docker compose up -d` usando la red compartida `pg_net`. En el master se instala `postgres_fdw`, se registran los workers como servidores foráneos y se crean las foreign tables que apuntan a las tablas físicas de cada worker:

```
services:
  pg_master:
    image: postgres:16
    container_name: pg_master
    environment:
      POSTGRES_PASSWORD: pass
    ports:
      - "5432:5432"
    networks:
      - pg_net
  pg_worker1:
    image: postgres:16
    container_name: pg_worker1
    environment:
      POSTGRES_PASSWORD: pass
    ports:
      - "5433:5432"
    networks:
      - pg_net
  pg_worker2:
    image: postgres:16
    container_name: pg_worker2
```

```

environment:
  POSTGRES_PASSWORD: pass
ports:
  - "5434:5432"
networks:
  - pg_net
networks:
  pg_net:
    driver: bridge

```

Desde el master se registran los workers y se crean los mapeos de usuario:

```

CREATE EXTENSION IF NOT EXISTS postgres_fdw;

CREATE SERVER worker1
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host 'pg_worker1', port '5432', dbname 'postgres');

CREATE SERVER worker2
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host 'pg_worker2', port '5432', dbname 'postgres');

CREATE USER MAPPING FOR postgres SERVER worker1
  OPTIONS (user 'postgres', password 'pass');

CREATE USER MAPPING FOR postgres SERVER worker2
  OPTIONS (user 'postgres', password 'pass');

```

Las foreign tables en el master mapean directamente a las tablas físicas de los workers:

```

CREATE FOREIGN TABLE AtencionMedica_Obesidad_fdw (
  DNI CHAR(8), CodMedico INTEGER, Ciudad VARCHAR(50),
  Diagnostico VARCHAR(50), Peso DECIMAL(5,2), Talla DECIMAL(4,2),
  PresionArterial VARCHAR(10), Edad INTEGER, FechaAtencion DATE
) SERVER worker1 OPTIONS (table_name 'atencionmedica_obesidad');

CREATE FOREIGN TABLE AtencionMedica_Cardiopatia_fdw (
  DNI CHAR(8), CodMedico INTEGER, Ciudad VARCHAR(50),
  Diagnostico VARCHAR(50), Peso DECIMAL(5,2), Talla DECIMAL(4,2),
  PresionArterial VARCHAR(10), Edad INTEGER, FechaAtencion DATE
) SERVER worker1 OPTIONS (table_name 'atencionmedica_cardiopatia');

CREATE FOREIGN TABLE AtencionMedica_Hipertension_fdw (
  DNI CHAR(8), CodMedico INTEGER, Ciudad VARCHAR(50),
  Diagnostico VARCHAR(50), Peso DECIMAL(5,2), Talla DECIMAL(4,2),
  PresionArterial VARCHAR(10), Edad INTEGER, FechaAtencion DATE
) SERVER worker2 OPTIONS (table_name 'atencionmedica_hipertension');

CREATE FOREIGN TABLE Pacientes_F2_fdw (

```



```

DNI CHAR(8), Nombre VARCHAR(50), Apellidos VARCHAR(100),
FechaNacimiento DATE, Sexo CHAR(1), CiudadOrigen VARCHAR(50)
) SERVER worker1 OPTIONS (table_name 'pacientes_f2');

CREATE FOREIGN TABLE Pacientes_F3_fdw (
DNI CHAR(8), Nombre VARCHAR(50), Apellidos VARCHAR(100),
FechaNacimiento DATE, Sexo CHAR(1), CiudadOrigen VARCHAR(50)
) SERVER worker2 OPTIONS (table_name 'pacientes_f3');

```

Q1. SELECT * FROM Pacientes ORDER BY FechaNacimiento

Cada fragmento se materializa en una tabla temporal ordenada por **FechaNacimiento**. El fragmento F1 es local en el master; F2 y F3 se obtienen mediante FDW desde los workers. El merge final se ejecuta en el coordinador.

```

CREATE TEMP TABLE r1 ON COMMIT DROP AS
SELECT * FROM Pacientes_F1 ORDER BY FechaNacimiento;

CREATE TEMP TABLE r2 ON COMMIT DROP AS
SELECT * FROM Pacientes_F2_fdw ORDER BY FechaNacimiento;

CREATE TEMP TABLE r3 ON COMMIT DROP AS
SELECT * FROM Pacientes_F3_fdw ORDER BY FechaNacimiento;

EXPLAIN ANALYZE
SELECT * FROM (
SELECT * FROM r1
UNION ALL SELECT * FROM r2
UNION ALL SELECT * FROM r3
) m
ORDER BY FechaNacimiento;

```

The screenshot shows a SQL IDE interface with a query editor on the left and a query plan viewer on the right. The query in the editor is the same as the one in the previous block. The query plan viewer shows the following details:

- Showing rows: 1 to 9 of 1
- QUERY PLAN text
- 1 Sort (cost=3220.02..3241.62 rows=8640 width=502) (actual time=24.334..31.071 rows=60000 loops=1)
- 2 Sort Key: r1.fechanacimiento
- 3 Sort Method: external merge Disk: 2864kB
- 4 -> Append (cost=0.00..705.60 rows=8640 width=502) (actual time=0.007..8.591 rows=60000 loops=1)
- 5 -> Seq Scan on r1 (cost=0.00..220.80 rows=2880 width=502) (actual time=0.007..1.715 rows=20000 loops=1)
- 6 -> Seq Scan on r2 (cost=0.00..220.80 rows=2880 width=502) (actual time=0.010..1.669 rows=20000 loops=1)
- 7 -> Seq Scan on r3 (cost=0.00..220.80 rows=2880 width=502) (actual time=0.009..1.560 rows=20000 loops=1)
- 8 Planning Time: 0.144 ms
- 9 Execution Time: 33.309 ms

Q2. SELECT DISTINCT CiudadOrigen FROM Pacientes

Cada worker aplica **DISTINCT** localmente sobre su fragmento. El master recibe únicamente las ciudades únicas de cada fragmento y las concatena con **UNION ALL**, sin necesidad de re-deduplicar porque la fragmentación

por **CiudadOrigen** garantiza que cada ciudad vive en un solo fragmento.

```
CREATE TEMP TABLE r1 ON COMMIT DROP AS
  SELECT DISTINCT CiudadOrigen FROM Pacientes_F1;

CREATE TEMP TABLE r2 ON COMMIT DROP AS
  SELECT DISTINCT CiudadOrigen FROM Pacientes_F2_fdw;

CREATE TEMP TABLE r3 ON COMMIT DROP AS
  SELECT DISTINCT CiudadOrigen FROM Pacientes_F3_fdw;

EXPLAIN ANALYZE
SELECT CiudadOrigen FROM r1
UNION ALL SELECT CiudadOrigen FROM r2
UNION ALL SELECT CiudadOrigen FROM r3;
```

The screenshot shows a database IDE with a dark theme. The left pane displays the SQL query from the previous block. The right pane shows the 'QUERY PLAN' for the query. The plan consists of six steps:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loops
1	Append	0.00..54.75	rows=1650	width=118	(actual time=0.005..0.012)	rows=18	loops=1
2	-> Seq Scan on r1	(cost=0.00..15.50)	rows=550	width=118	(actual time=0.004..0.005)	rows=6	loop...
3	-> Seq Scan on r2	(cost=0.00..15.50)	rows=550	width=118	(actual time=0.002..0.003)	rows=7	loop...
4	-> Seq Scan on r3	(cost=0.00..15.50)	rows=550	width=118	(actual time=0.002..0.002)	rows=5	loop...
5	Planning Time	0.083 ms					
6	Execution Time	0.020 ms					

Q3. SELECT Diagnostico, AVG(Edad) AS PromEdad FROM AtencionMedica GROUP BY Diagnostico

Cada fragmento calcula su propio **AVG(Edad)** de forma local (uno por diagnóstico). El master recibe exactamente una fila por fragmento y las concatena con **UNION ALL**. No es necesaria una re-agregación porque cada fragmento almacena un único valor de **Diagnostico**.

```
CREATE TEMP TABLE r1 ON COMMIT DROP AS
  SELECT Diagnostico, AVG(Edad) AS PromEdad
  FROM AtencionMedica_Diabetes GROUP BY Diagnostico;

CREATE TEMP TABLE r2 ON COMMIT DROP AS
  SELECT Diagnostico, AVG(Edad) AS PromEdad
  FROM AtencionMedica_Obesidad_fdw GROUP BY Diagnostico;

CREATE TEMP TABLE r3 ON COMMIT DROP AS
  SELECT Diagnostico, AVG(Edad) AS PromEdad
  FROM AtencionMedica_Cardiopatia_fdw GROUP BY Diagnostico;

CREATE TEMP TABLE r4 ON COMMIT DROP AS
  SELECT Diagnostico, AVG(Edad) AS PromEdad
```

```
FROM AtencionMedica_Hipertension_fdw GROUP BY Diagnostico;
```

```
EXPLAIN ANALYZE
SELECT * FROM r1
UNION ALL SELECT * FROM r2
UNION ALL SELECT * FROM r3
UNION ALL SELECT * FROM r4;
```

The screenshot shows a SQL IDE interface. On the left, a query is written in a dark-themed editor. On the right, the 'Query Plan' tab is active, displaying the execution plan for the query. The query plan shows a sequence of operations: Append, Seq Scan on r1, Seq Scan on r2, Seq Scan on r3, and Seq Scan on r4, followed by Planning Time and Execution Time.

```

39 CREATE TEMP TABLE r1 ON COMMIT DROP AS
40   SELECT Diagnostico, AVG(Edad) AS PromEdad
41   FROM AtencionMedica_Diabetes GROUP BY Diagnostico;
42
43 CREATE TEMP TABLE r2 ON COMMIT DROP AS
44   SELECT Diagnostico, AVG(Edad) AS PromEdad
45   FROM AtencionMedica_Obesidad_fdw GROUP BY Diagnostico;
46
47 CREATE TEMP TABLE r3 ON COMMIT DROP AS
48   SELECT Diagnostico, AVG(Edad) AS PromEdad
49   FROM AtencionMedica_Cardiopatia_fdw GROUP BY Diagnostico;
50
51 CREATE TEMP TABLE r4 ON COMMIT DROP AS
52   SELECT Diagnostico, AVG(Edad) AS PromEdad
53   FROM AtencionMedica_Hipertension_fdw GROUP BY Diagnostico;
54
55 EXPLAIN ANALYZE
56 SELECT * FROM r1
57 UNION ALL SELECT * FROM r2
58 UNION ALL SELECT * FROM r3
59 UNION ALL SELECT * FROM r4;

```

Query Plan:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Append	(cost=0.00..67.00)	rows=1800	width=150	(actual time=0.003..0.009)	rows=4	loops=1
2	-> Seq Scan on r1	(cost=0.00..14.50)	rows=450	width=150	(actual time=0.003..0.003)	rows=1	loop...
3	-> Seq Scan on r2	(cost=0.00..14.50)	rows=450	width=150	(actual time=0.001..0.002)	rows=1	loop...
4	-> Seq Scan on r3	(cost=0.00..14.50)	rows=450	width=150	(actual time=0.001..0.002)	rows=1	loop...
5	-> Seq Scan on r4	(cost=0.00..14.50)	rows=450	width=150	(actual time=0.001..0.001)	rows=1	loop...
6	Planning Time	0.087 ms					
7	Execution Time	0.019 ms					

Q4. SELECT * FROM Pacientes NATURAL JOIN AtencionMedica

Se aplica el algoritmo de semi-join distribuido en cinco fases. Primero se proyectan los DNIs presentes en cada fragmento de **AtencionMedica** (incluyendo los workers vía FDW). Luego se construye el conjunto unificado de DNIs y se filtra cada fragmento de **Pacientes** antes de enviarlo al master. Finalmente el coordinador ejecuta el join sobre los datos ya reducidos.

```

-- Fase 1: proyección local de DNI en cada fragmento de AtencionMedica
CREATE TEMP TABLE dni_a1 ON COMMIT DROP AS
  SELECT DISTINCT DNI FROM AtencionMedica_Diabetes;

CREATE TEMP TABLE dni_a2 ON COMMIT DROP AS
  SELECT DISTINCT DNI FROM AtencionMedica_Obesidad_fdw;

CREATE TEMP TABLE dni_a3 ON COMMIT DROP AS
  SELECT DISTINCT DNI FROM AtencionMedica_Cardiopatia_fdw;

CREATE TEMP TABLE dni_a4 ON COMMIT DROP AS
  SELECT DISTINCT DNI FROM AtencionMedica_Hipertension_fdw;

-- Fase 2: unión deduplicada de DNIs en el coordinador
CREATE TEMP TABLE dni_atencion ON COMMIT DROP AS
  SELECT DNI FROM dni_a1
  UNION SELECT DNI FROM dni_a2
  UNION SELECT DNI FROM dni_a3
  UNION SELECT DNI FROM dni_a4;

```

```
-- Fase 3: semi-join en cada fragmento de Pacientes
CREATE TEMP TABLE p_filt_1 ON COMMIT DROP AS
  SELECT * FROM Pacientes_F1
  WHERE DNI IN (SELECT DNI FROM dni_atencion);

CREATE TEMP TABLE p_filt_2 ON COMMIT DROP AS
  SELECT * FROM Pacientes_F2_fdw
  WHERE DNI IN (SELECT DNI FROM dni_atencion);

CREATE TEMP TABLE p_filt_3 ON COMMIT DROP AS
  SELECT * FROM Pacientes_F3_fdw
  WHERE DNI IN (SELECT DNI FROM dni_atencion);

-- Fase 4: reunir el lado izquierdo reducido en el coordinador
CREATE TEMP TABLE p_reduced ON COMMIT DROP AS
  SELECT * FROM p_filt_1
  UNION ALL SELECT * FROM p_filt_2
  UNION ALL SELECT * FROM p_filt_3;

-- Fase 5: consolidar AtencionMedica completa y ejecutar el join final
CREATE TEMP TABLE atencion_completa ON COMMIT DROP AS
  SELECT * FROM AtencionMedica_Diabetes
  UNION ALL SELECT * FROM AtencionMedica_Obesidad_fdw
  UNION ALL SELECT * FROM AtencionMedica_Cardiopatía_fdw
  UNION ALL SELECT * FROM AtencionMedica_Hipertension_fdw;

EXPLAIN ANALYZE
SELECT * FROM p_reduced JOIN atencion_completa USING (DNI);
```

The screenshot shows a PostgreSQL query editor with a query plan for the following query:

```
SELECT * FROM p_reduced JOIN atencion_completa USING (DNI);
```

The query plan details the execution of a Merge Join between `p_reduced` and `atencion_completa`. The plan includes the following steps:

- Merge Join (cost=5311.40..11181.80 rows=387072 width=812) (actual time=234.638..295.802 rows=60000 loops=1)
- Merge Cond: (`p_reduced.dni = atencion_completa.dni`)
- > Sort (cost=661.49..673.49 rows=4800 width=502) (actual time=44.190..45.955 rows=34904 loops=1)
- Sort Key: `p_reduced.dni`
- Sort Method: quicksort Memory: 3336kB
- > Seq Scan on `p_reduced` (cost=0.00..368.00 rows=4800 width=502) (actual time=0.038..5.085 rows=34904 loops=1)
- > Materialize (cost=4649.91..4730.55 rows=16128 width=346) (actual time=190.439..215.147 rows=60000 loops=1)
- > Sort (cost=4649.91..4690.23 rows=16128 width=346) (actual time=190.432..208.494 rows=60000 loops=1)
- Sort Key: `atencion_completa.dni`
- Sort Method: external merge Disk: 4232kB
- > Seq Scan on `atencion_completa` (cost=0.00..929.28 rows=16128 width=346) (actual time=0.053..13.287 rows=60000 loops=1)
- Planning Time: 0.121 ms
- Execution Time: 298.815 ms

Explicación de Resultados P5

Q1 — `SELECT * FROM Pacientes ORDER BY FechaNacimiento`

Plan:

- **Append** recorre las tres temp tables (**r1**, **r2**, **r3**), cada una con **20 000 filas** = **60 000 filas totales**. Los fragmentos resultaron balanceados porque la distribución de ciudades entre los tres rangos fue equitativa.
- El coordinador aplica un **Sort** final con **Sort Method: external merge** (Disk: 2864 kB), equivalente al k-way merge del algoritmo distribuido.
- **Execution Time = 33.309 ms**, significativamente mayor que los 14.18 ms de P4, porque la construcción de **r2** y **r3** implicó transferir 20 000 filas cada una desde los workers a través de la red Docker antes de que el EXPLAIN pudiera ejecutarse.

Q2 — SELECT DISTINCT CiudadOrigen FROM Pacientes**Plan:**

- **Append** sobre **r1 (6 filas)**, **r2 (7 filas)**, **r3 (5 filas)** = **18 ciudades distintas**, idéntico resultado al de P4.
- **Seq Scan** simple en cada temp table; no se requiere re-deduplicación porque la partición por **CiudadOrigen** garantiza que cada ciudad existe en un único fragmento.
- **Execution Time = 0.020 ms**. Ligeramente superior a los 0.012 ms de P4 por la latencia de red durante la construcción de las tablas temporales, pero igualmente sub-milisegundo para el EXPLAIN final.

Q3 — SELECT Diagnostico, AVG(Edad) GROUP BY Diagnostico**Plan:**

- **Append** sobre **r1**, **r2**, **r3**, **r4**, cada una con **exactamente 1 fila** = **4 filas finales**.
- Cada fragmento calculó su **AVG(Edad)** de forma local antes de ser materializado en la temp table, por lo que el coordinador solo recibe resultados ya agregados.
- **Execution Time = 0.019 ms**. Prácticamente idéntico a los 0.011 ms de P4 porque las temp tables intermedias son mínimas; la diferencia de red se absorbe en el overhead de construcción previo al EXPLAIN.

Q4 — SELECT * FROM Pacientes NATURAL JOIN AtencionMedica**Plan:**

- El planificador eligió **Merge Join** en lugar del Hash Join de P4, ordenando ambas entradas por **DNI**.
- **p_reduced** contiene **34 904 filas** (pacientes filtrados por semi-join), ordenadas con **quicksort en memoria** (3 336 kB).
- **atencion_completa** contiene **60 000 filas**, ordenadas con **Sort Method: external merge** (Disk: 4 232 kB), lo que indica que el volumen supera la memoria de trabajo disponible.
- **Execution Time = 298.815 ms**, muy superior a P4 porque la construcción de **atencion_completa** requirió transferir los 15 000 registros de cada fragmento remoto (Obesidad, Cardiopatía, Hipertensión) a través de la red antes del join final.

Diferencia clave respecto a P4

Aspecto	P4 (local)	P5 (distribuido)
Fuente de los fragmentos	Particiones locales en un único servidor	Tablas físicas en tres contenedores Docker
Mecanismo de acceso remoto	No aplica	<code>postgres_fdw</code> con foreign tables (<code>_fdw</code>)
Visibilidad en EXPLAIN del nodo final	<code>Seq Scan</code> sobre particiones locales	<code>Seq Scan</code> sobre temp tables (el <code>Foreign Scan</code> ocurrió en <code>CREATE TEMP TABLE</code>)
Q1 Execution Time	14.18 ms	33.309 ms
Q2 Execution Time	0.012 ms	0.020 ms
Q3 Execution Time	0.011 ms	0.019 ms
Q4 Execution Time	(local, ms bajos)	298.815 ms

Conclusiones

- **Q2 y Q3** siguen siendo los casos óptimos también en el escenario distribuido: la clave de la operación coincide con la clave de partición, por lo que cada worker entrega un resultado ya reducido al master.
- **Q1** evidencia el costo de la red: los 20 000 registros de cada worker deben transferirse íntegramente al master antes del merge, lo que más que dobla el tiempo respecto a P4.
- **Q4** es el caso más costoso en el escenario distribuido porque el semi-join, aunque reduce el volumen de `Pacientes`, no puede evitar que el coordinador consolide todos los 60 000 registros de `AtencionMedica` desde cuatro fuentes (una local y tres remotas) para ejecutar el join final.
- Los planes EXPLAIN del merge final muestran `Seq Scan` sobre tablas temporales en lugar de `Foreign Scan`, porque el acceso FDW se produce durante la fase `CREATE TEMP TABLE`. Sin embargo, el overhead de red queda reflejado en los tiempos de ejecución globales superiores a los de P4.